

CREATING A CLOUD-BASED NOTE-TAKING SYSTEM FOR PEOPLE WITH ADHD: IMPROVING ACCESSIBILITY WITH NEURODIVERSE-CENTERED DESIGN AND AI

CHAPTER ONE INTRODUCTION

1.1 Background of the Study

There has been increased interest in the intersection of neurodiversity and education technology in recent years, most prominently in meeting the distinctive learning needs of people with Attention-Deficit/Hyperactivity Disorder (ADHD). ADHD is described by its persistent patterns of inattention, impulsiveness, and restlessness and presents distinctive difficulties in the academic and work environments. Standard methods of taking notes are found wanting to cope with the variable attention and difficulties with executive functioning among people with ADHD (Heublein, 2023). As such, the adaptation and personalization of tools like cloud-based systems for taking notes provide an alternative to traditional approaches that are well suited to embedding principles of universal design for learning (Harrison et al., 2024). The systems are designed not merely to enhance accessibility but to create independence, decrease load on the brain, and support the neurodiverse needs of learners. The convergence of assistive design with cloud computing technologies has made it possible to develop smart, collaborative platforms that break through the constraints of conventional learning tools. Cloud systems offer capabilities like real-time syncing, multi-device support, and AI-enhanced features, which are helpful particularly in the case of ADHD users who need assistance with organization and retrieval of information (Smart et al., 2015). In addition to this, AI integration when taking notes—through automated summarization, highlighting keywords, and focus-sensitive input detection—can engage actively with users by minimizing the level of cognitive demand to write down and maintain notes efficiently (Sheerin et al., 2024). Access to these systems can dramatically enhance the learning experience of neurodiverse learners by providing structure, routine, and adjustability aligned with their learning profiles. There is still a significant discrepancy in the mainstream implementation of neurodiverse-focused design among digital learning tools. Numerous current note-taking tools are either not inclusive or not accommodating to individuals with ADHD (Shama et al., 2024). Studies reveal that ADHD students need systems that are attuned to their cognitive patterns, offer customizable interface components, and include prompts or reminders that support persistent engagement (OCAD et al., 2012). A cloud-based taking system that merges AI with inclusive design processes—such

as audio, voice, or touch inputs—offers a revolutionary way to solve these issues (Lee et al., 2023). As the digital learning landscape advances further, integrating accessibility from the bottom up becomes not only a technical imperative but an ethical prerequisite to education fairness.

1.2 Problem Statement

In spite of the swift progress in digital tools and learning technologies, adults with ADHD are still struggling with capturing, filtering, and remembering information in academic and work environments. Note taking traditionally fails to meet the particular cognition needs—such as attention regulation difficulties, working memory, and executive function—presented by individuals with ADHD, and thus fails to capture information, disorganizes notes, and increases anxiety. While there are some note taking apps existing today, many are built with an one-size-fits-some approach that fails to consider the variety through which humans access and use information. Thus, these products lack adaptive capabilities attuned to adapt to people's unique ways and needs when accessing and working with information. The lack of neurodiverse-inclined designing in mainstream products leaves the platform with an opportunity gap whereby neurodiverse users like those with ADHD are compelled to conform to the tools rather than the tools conforming to them. What is required is an adaptive system in the cloud that not only provides convenience and access to users, but is designed with an empathic knowledge regarding how people with ADHD work with information. Such a system should leverage AI not to merely do the work, but to make the user experience better with adaptive capabilities—such as smart prompts, audio and visual aids, real-time collaboration, and context-conscious reminders—that foster focus and retention of information. Without this type of thoughtful designing, tools will continue to under-serve neurodiverse users, effectively perpetuating the very hurdles that the tools are designed to eliminate (Khan et al., 2021).

1.3 Aims and Objectives

1.3.1 Aims

1. To create a cloud-based system for taking notes that is particularly intended to serve the cognitive requirements of individuals with ADHD through embedding features founded on principles of neurodiverse-centric design.
2. To utilize cloud based architecture to maximize accessibility and user participation with adaptive and intuitive tools that support users in controlling attention, memory, and organization capabilities.

1.3.2 Goals of Research

1. To examine the limitations of current note tools in meeting the needs of individuals with ADHD and assess how these inadequacies impact learning and productivity.
2. To investigate user-centered methods to support the inclusiveness and accessibility of neurodiverse users in particular, particularly in online learning environments (OCAD et al., 2012).
3. To develop and prototype a system in the cloud with easy-to-use tools to support adaptability in individuals with ADHD.
4. To validate the efficacy of the proposed system to enhance organization of notes and task management
5. To assist with the general conversation surrounding universal design for learning by exemplifying how technology can be made to adapt to neurodiversity instead of expecting users to fit into preset systems.

2. LITERATURE REVIEW

2.1 Understanding ADHD in the Context of Learning and Productivity

2.1.1 Cognitive Characteristics of ADHD

Attention-Deficit/Hyperactivity Disorder (ADHD) is well known to be a neurodevelopmental disorder that is experienced by individuals in all aspects of life and particularly in learning and information processing. Though stereotyped by superficial behaviors like restlessness and distraction, the intellectual difficulties related to ADHD are very deep and more frequently extend to executive functions, working memory, and the ability to sustain attention. They are most impactful in environments that require linear processing, structure, and self-regulation—like the conventional classroom or work environment. To illustrate this point, several people with ADHD are impaired in getting going on tasks, planning out ideas, and filtering out extraneous stimuli, thus making taking notes—usually an independent, focused task—particularly problematic (Heublein, 2023).

These intellectual characteristics are not deficits in and of themselves but are instead an alternative way of processing information. Individuals with ADHD are able to perform high-level creativity work and make intuitive leaps and see patterns, but in unlinear and dynamic patterns. Traditional learning tools and productivity systems are not designed to tap into these capabilities and instead punish individuals who break away from conventional workflow patterns. The difference in the way neurodiverse individuals think and the way digital systems are built together create an ongoing misfit that can cause decreased academic performance, lowered self-esteem, and more mental wear and tear.

In the context of note-taking, this mismatch becomes particularly evident. Individuals with ADHD might begin notes enthusiastically but fail to maintain consistency. Others might record vast amounts of unstructured information, later finding it unusable or overwhelming. This calls for more adaptive, flexible systems that are not just “accessible” in the broad sense but truly centered around the lived experiences and unique processing styles of those with ADHD. Technologies that provide visual scaffolding, chunking of tasks, reminders, or audio inputs can

serve as cognitive allies, bridging the gap between natural thinking rhythms and effective information capture.

Cognitive Challenge	ADHD Learners	Traditional Learning Tools
Working Memory	Struggles with holding multiple items in	Relies on long-form note-taking and static
Sustained Attention	Prone to distractions and mental fatigue	Requires prolonged focus for comprehens
Executive Function & Planning	Difficulty organizing thoughts/tasks	Linear, inflexible structure
Task Initiation	Delayed starts; overwhelmed by comple	Lacks prompts or adaptive scaffolding
Sensory Sensitivity	Easily overstimulated or under-responsi	Minimal sensory customization or adaptal

Figure 1: Comparison of cognitive challenges faced by ADHD learners and limitations of traditional learning tools.

2.1.2 Common Learning Barriers in Note-Taking

Note-taking itself is essentially an exercise in multitasking with one's brain. It demands listening and processing and organizing and writing—and doing all these simultaneously in real-time. In individuals with ADHD, this becomes an unpredictable loop of split attention, fragmented notes, and confusion afterward. Probably one of the most prevalent problems is failing to sift through important matter during lectures or meetings. Instead of selective copying down the important points, individuals with ADHD might attempt to write down everything or nothing and sometimes even both, depending on focus (Chudaeva, 2024). Not only does this lead to erratic notes, but also greater disengagement from the material being taught itself.

Furthermore, most current note-taking tools assume some level of executive function—such as to assign tags to notes, to file them properly, or to look them up in the future. Neurodiverse learners will find these extra maneuvers to be significant hurdles. The tools meant to enhance learning instead create extra friction and make it more taxing in the brain rather than easing it. Paper methods are simple but don't support features like search or instant access on any device, and most software solutions provide too much complexity without smart assistance or customization.

Note-Taking Cycle and ADHD-Specific Cognitive Disruptions

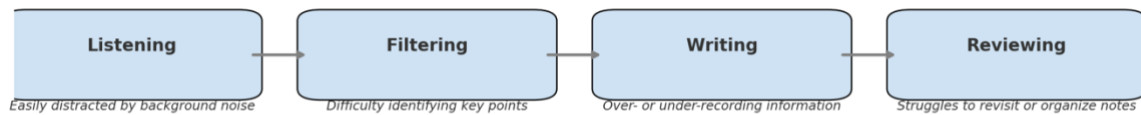


Figure 2: Visual representation of the note-taking process with ADHD-specific cognitive disruptions at each stage.

This is exacerbated in environments that emphasize speed and focus, like high-speed lectures or meetings. Without access to real-time assistance like voice-to-text, smart prompts, or chunked visual display, ADHD learners lag behind, feeding into anxiety and disengagement. We can clearly see that making tools to aid in this type of note-taking needs to extend beyond interface and feature design; it must include empathizing with neurodiverse work patterns.

2.1.3 Need for Assistive Technologies in Cognitive Support

As there is greater acknowledgement of neurodiversity in the education system and the workforce, there is an expanding need for assistive technologies that do more than offer surface-level support. True support needs to address the way that individuals think, recall, and engage with information in everyday life. Assistive technologies intended to address ADHD can revolutionize how individuals receive content, complete tasks, and memorize material. Those tools need to be more than “accessible” in the compliance sense—they need to be intuitive, interactive, and user-centric and focused on user agency and control (OCAD et al., 2012).

One potentially fruitful direction is through AI-powered systems that are able to identify attention drifts, provide recommendations for content abstracts, and structure notes with context and behavior in mind. Adding voice input with keyword extraction to support users who think in words, for example, can assist those who think out loud. Visual prompts, timers, or soft prompts can provide attentional control without taxing the user too much. Such

“cognitive offloading” enables ADHD people to conserve scarce executive bandwidth for comprehension and contemplation instead of mechanical operations like formatting and structuring information.

Smart Features in Assistive Note-Taking System

AI Summarization

Auto-extracts key ideas from long texts

Speech-to-Text

Transcribes spoken content into written notes

Reminders & Prompts

Helps users stay on task with timed nudges

Visual Organization

Organizes notes using layouts, colors, or maps

Figure 3: Smart assistive note-taking tools use AI to enhance accessibility and user focus.

What becomes evident is that whereas diagnosis of ADHD entails some cognitive limitations, technology—if deliberately designed—can be an enabling influence. The possibility to create adaptive systems that learn from the user, adapt in real-time, and provide the most cognitively palatable forms of presentation is immense. Such systems can transcend mere convenience to be an empowering tool to neurodiverse users.

2.1.4 Emotion Regulation and Cognition Overload in ADHD

Learning is both an intellectual task and an emotional experience for individuals with ADHD. They perceive and react more intensely to academic demands on an everyday level when the task appears overwhelming or hard to begin. In addition to the intellectual difficulties like disorganization or mental exhaustion, which are usually accompanied by the difficulties in

managing feelings like frustration, irritability or anxiety (Shaw et al., 2014), these feelings become hurdles to learning effectively, particularly when it needs to involve long-term attention and multitasking—such as in customary note-taking environments.

A second common experience is one of cognitive overload. Due to the tendency for ADHD to impact working memory and the ability to prioritize information, users can find themselves swamped by the sheer magnitude or speed of incoming data (Barkley, 2015). In the context of note taking, this can lead to writing down all the information simultaneously and thus leaving out important points or not taking notes under pressure. Such incidents don't only affect performance academically—they erode motivation and confidence with the passing of time.

Most current tools disregard this loop of emotion and cognition. They emphasize functionality—such as tagging, storage, or syncing—without taking into account how the learner is going to feel when using the tool. However, users with ADHD need emotional support. A reassuring visual interface, judicious prompts, and well-timed pauses can alleviate stress and enhance resilience. Certain systems are incorporating emotion-sensitive design and using AI to identify patterns of disengagement and trigger breaks or simplification (Pantic et al., 2020). They demonstrate it's not merely about minimizing cognitive load, but about assisting users to manage the emotional aspect of learning.

Ultimately, assisting ADHD is not just about providing tools—it is about establishing spaces that are feel-safe, attuned to the individual's needs, and human-focused. An effective note-taking system should not only support users to become organized but to feel perceived, validated, and empowered—even when battling through mental haze or anger.

2.1.5 The Design Considerations in ADHD-Focused Learning

Designing products or software to meet the needs of ADHD users does not simply depend on aesthetics or on general user testing—rather, it takes into account a sophisticated recognition of how neurodiverse people perceive, engage with, and react to messages. Unlike the neurotypical user group, individuals with ADHD usually perform well with systems that embrace impulsiveness, eliminate distraction and extraneous visual and auditory inputs, and make it easy to initiate and complete tasks. More and more literature focuses on the need to adapt interface and interaction to these particular cognitive and behavioral characteristics (de Barros et al., 2020). To build effective digital tools for ADHD, designers and developers should integrate the following five user-focused principles:

1. **Simplified Interfaces:** Minimize visual distractions with fewer on-screen elements. A decluttered and minimalist interface prevents distraction and allows users to maintain focus and prevent overstimulation, one of the main causes of disengagement (Lindstedt & Umbach, 2021).
2. **Chunking and task segmentation:** Divide big tasks into smaller mobile-friendly chunks. Micro-tasking aids in supporting executive function through presenting consumers with small, obtainable goals, minimizing feelings of being overwhelmed, and increasing motivation (Weyandt et al., 2013).
3. **Real-Time Feedback and Support:** Add progress indicators, completion milestones, and timely prompts. Those reinforce desired behavior, provide emotion validation, and prevent abandonment to tasks.
4. **Multimodal Input and Output:** Provide several methods to interface with the system—voice, text, audio, visuals—whereby users are able to engage with the system in a way that fits their cognitive capabilities and needs (Puckett, 2024).
5. **Adaptive Learning Paths and Personalization:** Support customization of layout, font, colors, and reminder options. An adaptive system that learns and adapts to user behavior and tempo enhances feelings of control and mastery (Holmes et al., 2019).

These design considerations are not merely optional—they are foundational to creating inclusive, empowering tools that accommodate ADHD learners. As such, any serious development of a note-taking system for this population must embed these principles into both its functionality and user experience design.

2.2 Evolution of Note-Taking Technologies

2.2.1 From Paper to Digital: The Shift in Learning Tools

The evolution of note-taking parallels the general progress of learning and productivity tools. From pen and paper to spiral notebooks to artificial intelligence-empowered virtual environments, advances with each move in technology increased capability—though with increased design complexity in the bargain. Early on, note-taking was bound to pen and paper with the limitations of simplicity and little organization flexibility. Students and professionals had to use memory and manual sorting and categorization alone. The advent of word processors like Microsoft Word made it more efficient to type and edit text, but these tools were static and without connection or smart assistance (Schepman et al., 2012).

The emergence of note-taking software such as Evernote, OneNote, and Notability revolutionized this. They provided organizational capabilities—tags, folders, multimedia input—that allowed people to work with the information more fluidly. Yet these tools depended on human input and intended organization, and this usually created difficulties for individuals with ADHD. As mobility and integration increased with user demand, systems began to exist on the cloud, which allowed devices to sync in real-time and edit collaboratively. It addressed important issues with data loss and access but did not naturally cater to neurodiversity.

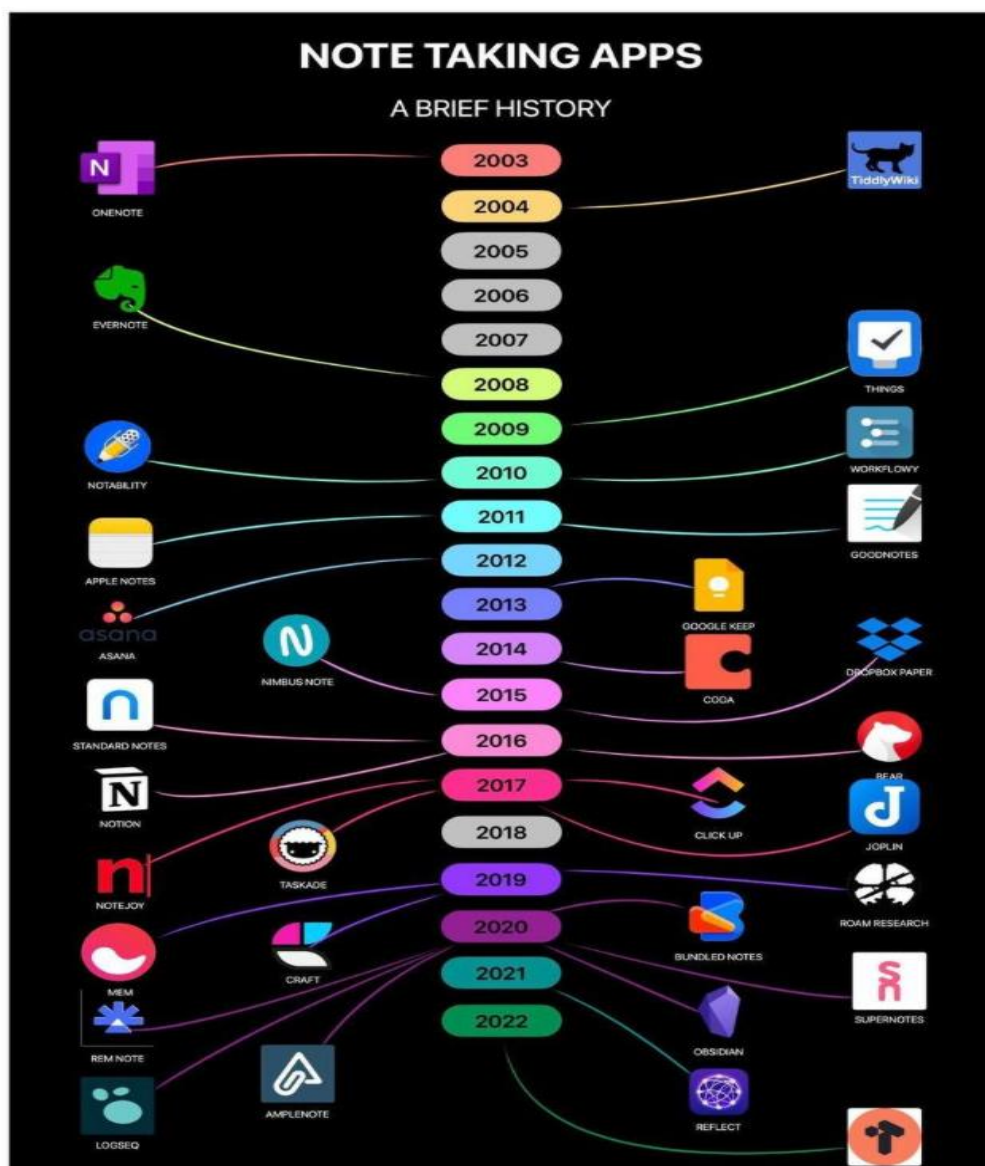


Figure 4: Evolution of note taking apps (photos.io)

2.2.2 Intelligent Note-Taking Systems and AI Integration

As digital ecosystems evolved, attention shifted to creating smart, context-sensitive note-taking systems. Such platforms used artificial intelligence to carry out actions like automated transcription, semantic summarizing, keyword extraction, and voice commands. Among neurodiverse users, this offered an innovative paradigm whereby the tool adjusted to the individual and not vice versa (Wang et al., 2022). Specifically, tools like PhenoPad and GAVIN showed the effectiveness of integrating AI to support greater user involvement. PhenoPad takes advantage of real-time voice and handwriting input to record clinical notes in clinical settings, and GAVIN employs both gaze and voice input to make taking notes more user-friendly. Both illustrate that AI can function as a scaffold to cognition—lessening the load on memory, organization, and focused attention, all skills with which users with ADHD struggle.

The following are some typical characteristics of AI-embedded note-taking tools:

1. **Speech to-Text Transcription:** Transcribes spoken words into typing-friendly text and eliminates the stress of typing.
2. **Contextual Summarization:** Pinpoints important themes and summarizes content into bite-sized chunks.
3. **Task Scheduling and Recognition:** Transforms action items from notes into deadline-based tasks.
4. **Real-Time Collaboration:** Enables simultaneous interaction with the same note environment by several users.

These developments lay the foundation for tools that are not only accommodating to alternative ways of thinking but also foster increased autonomy and effectiveness in learning.

2.2.3 Collaborative Note-Taking and Cloud-Based Environments

Note-taking on the cloud opened up an innovative dimension—boundaryless accessibility. Storing data on the cloud allowed users to access notes on any device, work from anywhere, and prevent data loss. Google Keep, Notion, and Apple Notes adopted syncing on the cloud and collaboration, increasing productivity and dependability enormously (Orndorff III, 2015).

As an integral part of this revolution, AWS (Amazon Web Services) has been pivotal in facilitating many contemporary applications with cloud computing resources. Serverless

computing, particularly AWS Lambda, enables developers to execute backend code on demand related to events (such as a user dictating into an app designed to take notes) without provisioning or managing servers. Such architecture is suited to those systems requiring high scalability and real-time responsiveness.

AWS Lambda can be applied in note-taking systems to:

1. Pipeline audio streams to transcribe in real-time
2. Trigger text summarization with AI when there is fresh content added
3. Synchronize and track data among users and devices in real-time

Moreover, Amazon DynamoDB—a highly performant, NoSQL database service—tends to be used with Lambda to hold structured note data. The pairing is particularly well suited to support lightweight, high-performance apps that are scalable by demand without sacrificing user experience.

Other platforms provide competitive alternatives too. Azure Functions and Azure Cosmos DB offer equivalent capabilities in Microsoft Azure, and Google Cloud Platform (GCP) uses Firebase to operate databases in real-time and use Google AI to process input. Yet, AWS is the most widely used because it is more mature and offers far-reaching integration capabilities (Kipps & Jones, 2022).

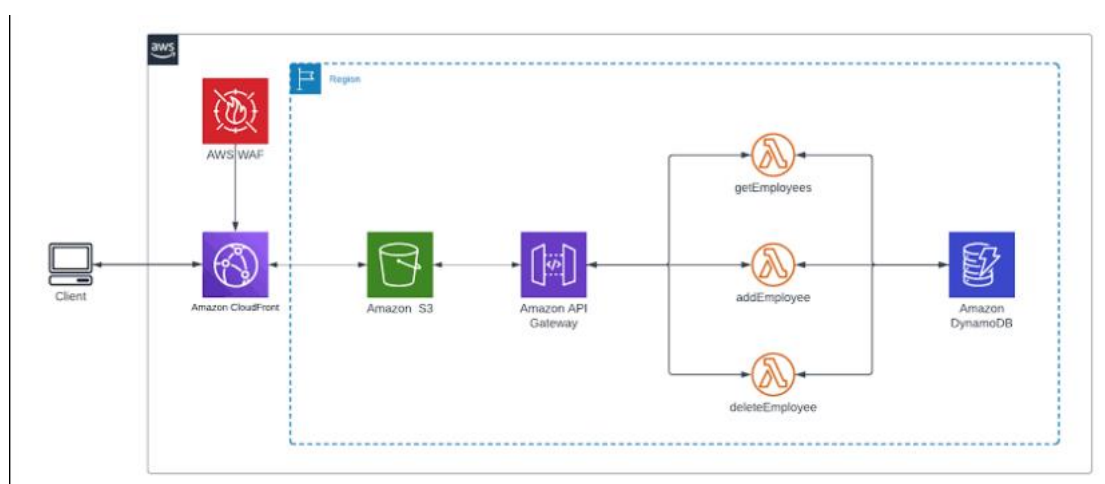


Figure 5: Serverless Note Architecture with AWS Lambda and Dynamo DB Kemane Donfack (2023)

The possibilities for these technologies in ADHD-friendly contexts are significant. Automating backends and taking tasks away from the user reduce friction and enable more streamlined cognitive workflows. Note taking's future is not only to capture information, but to free the user—particularly people with differences in cognition—to connect with it in useful and individual ways.

2.2.4 Smart Pens and Wearable Input Devices Integration

As note-taking technologies evolve, one promising area that has opened up is in conjunction with smart pens and wearable devices, specifically suitable for neurodiverse individuals who need text and kinesthetics to work most effectively. Smart pens like Livescribe or Neo Smartpen merge the analog with the digital by making the capture and storage of handwritten notes synchronized on the cloud platforms in real-time. In the case of individuals with ADHD, this provides traditional writing with the advantages of organization and retrieval through the use of digital tools (Sheerin et al., 2024).

Wearable input devices like smartwatches and AR glasses take note-taking into more mobile and interactive environments. They can offer ambient prompts, enable covert audio recording, or even support voice-controllable note commands—enabling users to capture thoughts on the move or when multitasking. Such adaptability is necessary for ADHD users who frequently experience delayed task implementation or forgetfulness if ideas are not captured immediately (Numazawa et al., 2014).

A significant advantage of these technologies is that they can decrease the frictions surrounding ideation and capture. Yet, most current devices are not neurodiverse-friendly. They are typically devoid of adaptive feedback systems, user-level customization options, or emotion-sensitive interfaces. Moreover, the costs and setup complexity could prevent access to these devices by many people. Nevertheless, the combination of haptic, wearable, and cloud-synchronized systems offers an enticing direction towards more investigation and development—most particularly towards creating inclusive and instant learning environments.

2.2.5 Customization and Interoperability in Contemporary Note-Taking

As the online world continues to grow, personalization and continuity between platforms are no longer niceties—it's necessary. It becomes particularly important to this population with

ADHD. Cognitive load, executive dysfunction, and attentional changes commonly cause single-device systems to be frustrating or worse, if not usable at all. Cross-platform compatibility and customization are thus necessary to enable neurodiverse users to stay on task without added cognitive load (Heublein, 2023).

The most efficient note-taking tools are those that provide smooth shifts—with devices, input methods, and learning environments alike. Whether on a smartwatch, smartphone, or desktop, the system needs to act similarly and provide flexible points of entry into the learning process. In addition to this, systems need to recall how users prefer to work and provide adaptive layout options, theme options, and even input options that adapt to changing focus during the daytime (Holmes et al., 2019).

2.2.5.1 Cross-Platform Synchronization and Accessibility

Most ADHD learners transition from context to context quickly—from school to home, from laptop to cell phone, or from keyboard to voice input. Without data syncing and interface consistency among these tools, the user experience is fractured. Cloud-based systems with real-time syncing among operating systems and form factors can eliminate the drag of note taking to a very great extent (Smart et al., 2015). Notion, Microsoft OneNote, and Google Keep are modern solutions with this level of adaptability to some extent. Not all software systems are equally functional on both Android and iOS, or equally retain data structure when accessed from one system to an alternative one. Such issues are particularly frustrating to ADHD individuals who might become discouraged by unanticipated layout and navigation changes.

What are required are systems that are designed with interoperability by default—not an afterthought. Such tools should:

1. Offer uniform feature sets on all platforms to prevent confusion or loss of features.
2. Ensure consistent layout to avoid having to relearn interactions on each device.
3. Provide real-time syncing support even under low-bandwidth environments to avoid data lag.
4. Allow offline access with auto-uploading once connectivity returns.
5. Save user customization options (font, contrast, input mode) in the cloud to use universally.

These factors minimize the cognitive load related to tool switching and enable ADHD users to maintain their state of cognitive flow without having to "reset" whenever tools are switched (Dodda et al., 2024).

2.2.5.2 Personalization as Cognitive Empowerment

As important to accessibility is the ability to customize. While numerous note-taking software tools provide cosmetic personalization—such as themes or font changes—relatively few systems extend beyond superficial options to enable functional customization. ADHD users will frequently need greater control: being able to fold away bothersome functionality, alter structuring format options on notes, or modify reminder delivery (Puckett, 2024).

Effective personalization allows cognitive empowerment by enabling users to customize the system to their individual learning cycles instead of having to adjust to the system itself. A user could have audio input when attention is poor in the morning and migrate to ordered templates when there is better mental acuity in the afternoon. The system can either anticipate these needs or make them easily configurable by hand (Lee et al., 2023).

Key personalization features that support ADHD learners include:

1. Toggleable interface complexity (simple vs. advanced mode)
2. User-defined task chunking templates
3. Notification frequency and tone adjustability
4. Smart font and contrast recommendations by time of day
5. Individualize voice command training to capture notes

If personalization is applied profoundly and meaningfully to people, it takes one-size-fits-all platforms and converts them into intellectual companions. It's not just an issue of accessibility—it's an issue of dignity and inclusion.

2.3 Cloud-Based Systems for Inclusive and Accessible Learning

2.3.1 Cloud Infrastructure for Educational Applications

Cloud computing has, in effect, transformed the way learning tools are created, deployed, and accessed. In education software—especially software designed to service neurodiverse learners—the cloud provides the agility, scalability, and uniformity that are critical to making adaptive, individualized learning experiences work. This is particularly beneficial in supporting ADHD individuals who are aided by software that is continuously available whenever and

wherever they need it, syncs instantaneously across their devices, and can update in real-time to meet their requirements (Compton & Allen, 2018).

For users who keep toggling between sites and devices, cloud infrastructure removes the exasperations of overhead backups, syncing hell-and platform lock-in. Also, cloud-based systems are very friendly to plug with smart learning capabilities like predictive analysis, NLP capabilities or smart notification that users require to be in-the-loop and organized. These abilities would be hard/impossible to do for an extended amount of time with legacy in-person or wired software models.

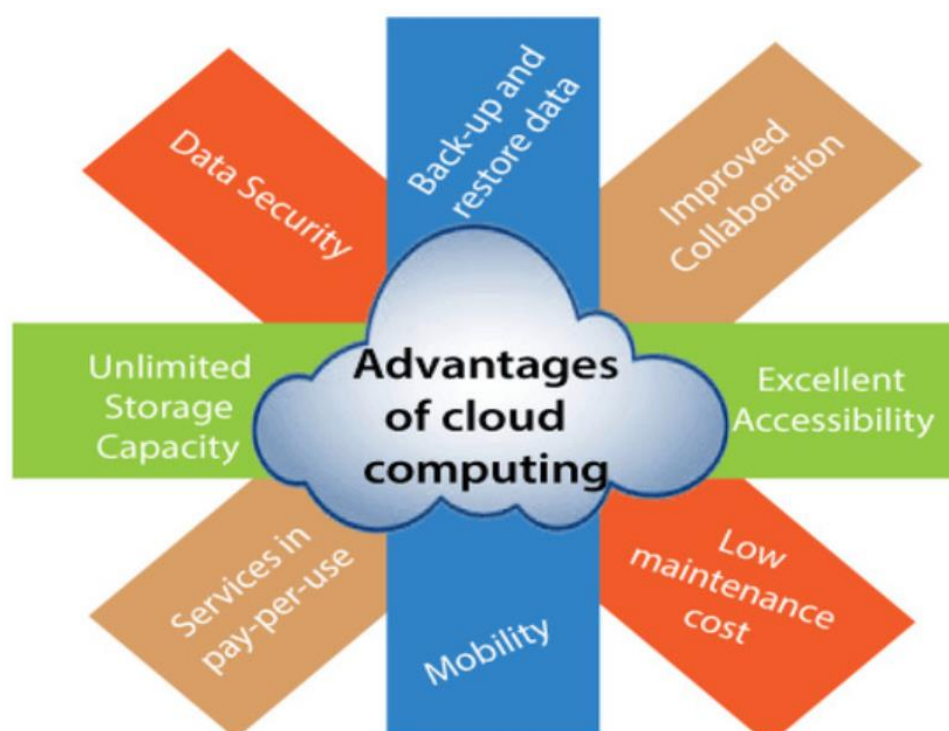


Figure 6: Cloud Computing Advantages Osama Abied et al. (2022)

2.3.2 Serverless Architectures in EdTech: AWS Lambda and Beyond

One of the most revolutionary technologies in application development in the cloud is serverless computing—a model of execution whereby developers write and deploy code without any need to manage infrastructure. In this model, cloud providers perform all the server administration, scaling, and resource allocation, thus enabling teams to focus on just the application function.

Among serverless solutions, AWS Lambda is one of the most popular with its scalability and compatibility with the overall AWS platform. Lambda allows for functions to be activated by designated events like saving an entry, submitting audio to be transcribed, or seeking an abstract to a lecture. Instant response is very helpful to ADHD users who require instant feedback or input to maintain interest.

The major benefits of AWS Lambda in ADHD-related applications are:

1. Event-Driven Execution – Only functions execute when required, minimizing resource utilization and expense.
2. Microservices Flexibility – It allows developers to create independent, modular features like voice transcription or auto-highlighting.
3. Real-Time Responsiveness – Facilitates instant processing of notes and delivery of feedback with little latency.
4. Unlimited Scalability – Seamlessly supports bursts in activity like in the midst of a live lecture or collaborative notes session.

These aspects enable one to develop an adaptive and smart system for taking notes that develops with the evolving needs of the user, without any performance bottleneck or user-level complexity (Smart et al., 2015).

AWS services such as Amazon DynamoDB complement Lambda by providing high performance and low latency storage for structured data like tagged notes, user account information, or accessibility options. DynamoDB does not need any user configuration and scales automatically, which means performance will never diminish with increases in user base.

2.3.3 Comparing Cloud Platforms: AWS vs. Azure vs. Google Cloud

Although AWS is a front runner in cloud-based learning infrastructure, it is not alone in this arena. Microsoft Azure and the Google Cloud Platform (GCP) are powerful options available as well, and each has features that can appeal to individual design sensibilities or learning approaches.

Major Considerations when Choosing a Platform:

1. AWS is the way to go if you need deep customization, high availability and integration with IoT and AI alike. It's also the most mature when it comes to supporting education platforms.
2. Azure would work best in institutions that utilize existing Microsoft software such as Teams, Office 365, or SharePoint. It is strong in emotion and sentiment detection through its Cognitive Services.
3. GCP is well suited to deploy mobile and real-time applications quickly and is thus ideal for dynamic user-facing functionality like collaborative editing or chat.

Ultimately, this will be driven by the particular aims of the system, available technical skills, and the final users' preferences. Nevertheless, the support offered by any one of these platforms to enable modularity, smartness, and scalability makes them particularly suitable to develop inclusive learning tools that genuinely address neurodiverse populations.

2.3.4 Ethical Considerations in Cloud-Based Learning Tools for Neurodiverse Users

As more and more cloud technologies become integrated into learning environments, ethical implications of using these technologies—and particularly among neurodiverse individuals—have become increasingly significant issues. Products intended to support individuals with ADHD frequently gather behavioral data, monitor patterns of engagement, and provide adaptive capabilities based on user interaction. Although these technologies can tailor learning and make it more accessible, they pose issues regarding privacy, consent, bias, and the use of data in the long term (Holmes et al., 2019).

Individuals with ADHD are already more vulnerable in school or work environments and thus ethical design becomes an important requirement. Cloud solutions need to not only adhere to legal frameworks such as the General Data Protection Regulation (GDPR), or the Family Educational Rights and Privacy Act (FERPA), but to be designed with transparency and user empowerment integrated into all aspects of the designing process. The following are some main ethical principles that need to inform ADHD-centered cloud-based note-taking systems' development and implementation:

1. **Informed Consent and Data Transparency:** Users must know clearly what data is being recorded, why it is being recorded and how it will be used. Consent

procedures need to be streamlined and made user-friendly for neurodiverse consumers.

2. **Minimization of Data Collection:** The system should collect only data required to function and not engage in invasive monitoring or heavy behavioral logging with no evident user advantage.
3. **Bias-Free AI Algorithms:** AI elements should be tested to prevent them from perpetuating assumptions and stereotypes about neurodiverse behavior. Systems should be trained on individual users rather than generic models.
4. **Right to Customize and Opt Out:** Users need to be able to modify or turn off specific AI capabilities (such as tracking-based prompts) without sacrificing underlying functionality.
5. **Long-term Data Stewardship:** Systems must include well-defined data retention and deletion policies so that learners own and control their learning history.

Ethical principles aren't secondary options—they are foundational to inclusive design. Integrating these principles into cloud-based tools supporting ADHD allows developers to promote not just access and usability, but also trustworthiness, agency, and psychosocial safety among neurodiverse individuals.

2.3.5 Emotional-Aware Interfaces and Neurofeedback in Cloud-Based Learning Systems

Among the directions to follow in systems designed for learning in the cloud are the incorporation and integration of neurofeedback and emotion-sensitive interfaces just beginning to be researched. In ADHD learners, emotion regulation is not secondary to learning and being able to engage with and respond to information (Shaw et al., 2014). Early evidence shows that learning spaces that are able to read emotion and dynamically adjust interface behavior can dramatically reduce incidents of frustration, mental burnout, and disengagement. Cloud-enabled systems can potentially incorporate emotion-sensing AI by way of camera input, typing behavior patterns, or wearables' sensor signals. The data can be used to adapt user interfaces in real-time: dimming the screen when agitation is detected, holding back stimulus when there are signals of being overwhelmed, or providing motivational cues when attention flags. Micro-adjustments make passive systems empathic spaces—those which “listen” rather than “teach” (Pantic et al., 2020). And neurofeedback—particularly when combined with cloud analytics—allows for more deep individualization. Sensors such as EEG headband can track levels of focus and send input back into the system to inform optimization in delivery of

material or trigger breaks when attention drops off. Recorded and analyzed in the cloud, these data offer the ability to longitudinally observe patterns in focus, weariness, and engagement on weeks and months. Such capabilities make real-time interaction and awareness of the self allowing users to manage attention with more sovereignty. But these developments tip responsibilities elsewhere: developers need to provide data protection and transparency on matters of consent and unobtrusive design. Insertion into cloud computing systems of emotion and cognition sensing is an important turning point in neurodiverse learning engineering. In listening to and attuning to users' feelings in the right way, these spaces are able to move the act of taking notes from having been purely intellectual to being psychologically secure and adaptive. In so doing, the spaces support not only knowledge retention but resilience, mastery and long-term learning success (Puckett, 2024).

2.3.6 Strategies of Gamification and Motivation in Cloud-Based Learning Systems for ADHD

While accessibility, emotion awareness, and real-time adjustment are paramount to inclusive learning systems in the cloud, sustained motivation is an equally important factor that is sometimes neglected. For those with ADHD—where an overwhelming majority struggle with motivation dysregulation and task aversion—adopting elements of gamification within learning platforms can make passive or otherwise overwhelming experiences enjoyable and rewarding ones (Lindstedt & Umbach, 2021).

The application of game principles outside gaming to enhance engagement, consistency, and satisfaction is called gamification. In environments that are cloud-supported and ADHD-friendly, this can come in numerous ways:

1. **Progress Monitoring and Achievement Badges:** Visual signals of task accomplishment or time-on-task can stimulate dopamine-facilitated feedback loops, enriching persistent engagement.
2. **Personalized Rewards and Goals:** Systems can provide micro-rewards—such as unlocking themes, gaining “focus points,” or activating ambient animations—for reaching some predetermined number of tasks or notes.
3. **Streaks and Soft Challenges:** As with fitness apps, note-taking systems can prompt users to create focus streaks or complete soft challenges that are suited to the user's thought rhythm.

4. Adaptive Goal-Setting: AI platforms can adjust difficulty or size of goals dynamically based on user behavior in the past to avoid overwhelming users and foster feelings of accomplishment (Puckett, 2024).

Gamification does not just entertain—it externalizes motivation for people with initiation issues with intrinsic motivation. Notably, the use of the cloud makes this more effective by keeping all progress synced and consistent when users move from one context to another (i.e., from home to school).

Critically, ADHD learners' gamification must be encouraging rather than coercive. Mechanics should foster feelings of autonomy rather than trigger anxiety through the threat of competitive comparisons and unbending schedules. Emotional-safety environments like opt-in contests or task-reduction options are critical.

Through the union of motivational design and real-time syncing with the cloud and adaptive analytics, developers are able to build not only usability—but learner resilience that lasts. Gamification therefore becomes an essential added benefit to making cloud systems not just accessible—but empowering as well.

2.4 Review of Related Works

Zhuang (2024) offers an efficient note-taking tool designed with AI integration for smart classrooms. Its strength is its emphasis on real-time summarization and extraction of content. Its weakness, however, is not having any interface options aware of neurodiversity. Comparative performance testing on devices is used in its methodology. Technologically sound in approach, the study fails to address issues regarding cognitive accessibility and instead leaves a gap in user-centered customization for those with ADHD (Zhuang, 2024).

Heublein (2023) offers an initial study on digital knowledge management designed for ADHD learners. A major strength is its user-centered design approach using surveys and iterative participant feedback from ADHD learners. But it does not explore real-time cloud syncing and AI-powered tools. Its qualitative methodology showed significant points to consider, but left unanswered questions on scalability and how to implement the cloud (Heublein, 2023).

Smart et al. (2015) created an extensible note-taking application that used cloud services and offline encrypted storage. The advantage of this work is its secure and platform-independent

architecture. The drawback is that it does not include AI-grounded cognitive capabilities. The authors applied a technical design and prototyping approach. Nevertheless, the work in this study did not touch on inclusiveness or personalization to the user level, leaving the possibility open to include neurodiverse-oriented improvements (Smart et al., 2015).

Chudaeva (2024) examines the note-taking practices of online learners through postphenomenological study. Its strength is interpretive depth to uncover emotional and sensory aspects of taking notes. It does not offer technological intervention and study ADHD needs, though. The study applied qualitative and experiential methodology. The lack of practical system designing or digital improvement points to an important area of study in translating findings into functional and inclusive implementation (Chudaeva, 2024).

Shama et al. (2024) provided an extensive overview of existing task and note-taking apps. Its power comes in the form of emphasis on varied features from app to app on platforms. It does not provide any focus on accessibility issues in ADHD users. Comparative analysis has been employed in the review without discussing user profiles from cognitive and behavioral standpoints. In assessment of inclusiveness or flexibility in task interfaces in neurodivergent individuals, there is a huge gap (Shama et al., 2024).

OCAD et al. (2012) provide design-focused recommendations for assistive technologies targeting ADHD students. Its major strength is its foundation in educational psychology and user behavior. However, it lacks integration with contemporary cloud or AI tools. The team used case studies and heuristic analysis. While deeply relevant to accessibility, it predates many technological innovations, presenting a gap in cloud-native and intelligent assistive tool development (OCAD et al., 2012).

Wang et al. (2022) created PhenoPad, an interface that uses AI to take notes in the medical field. Its major advantage is that it provides unified integration of real-time input processing and speech recognition. Field testing in clinical environments was used in methodology. While successful, it has little adaptability to be used by general learners or by neurodiverse students. The limitation is that it is single-purposed with no personalization capabilities to support cognition or inclusive design (Wang et al., 2022).

Lee et al. (2023) presented TacNote, which is an access tool with a haptic-audio note-taking system for the visually impaired. Its major advantage is its multimodal feedback system with

haptic and audio inputs. The prototype tested its usability. Though inclusive, it focuses on blindness rather than ADHD. It does not provide support against cognitive fatigue or against executive function obstacles, thus creating an area of need for systems tapping into varied sensory-cognitive learning needs (Lee et al., 2023).

Khan et al. presented GAVIN in 2021, an interface with gaze and voice together to capture notes implicitly. Its main strength is its new multimodal interaction, which minimizes human effort. Experimental user testing and eye-tracking data were used in its methodology. Though it is considered innovative, it does not support real-time editing and feedback capabilities required by ADHD users or include extensive user adaptability discussions, which indicates an area in neuroinclusive application design (Khan et al., 2021).

Puckett (2024) examines assistive writing technologies using the Universal Design for Learning paradigm. Although it corresponds with inclusive pedagogy, the research universalizes learning difficulties without separating ADHD-related needs. It is pedagogically relevant in its strength with toolkits facilitating learner autonomy. Yet it is devoid of technical richness, live options, and individualization. Cloud and AI are not discussed; hence, its suitability to build adaptive ADHD-focused note-taking tools is compromised.

Paek and Fulton (2016): The study assesses tablet-based tools to take notes among elementary school students in science class. The authors point to enhanced engagement and recall with digital interfaces. But it does not include neurodivergent learners and does not offer support strategies for ADHD users. Its strong point is its experimental classroom setup. But the tools employed are devoid of adaptive features or executive function assistance, rendering the findings inadequate for inclusive, ADHD-focused note-taking practices.

Saxena (2022) suggests augmented mind maps to extend cognitive organization. Visual mapping provides adaptive learning frameworks, a hypothetical benefit to ADHD users who are aided by non-linear thought processes. The work is missing an interface with cloud systems and does not account for sensory overload or tailoring to the individual. It is conceptually promising, but with no empirical ADHD-specific testing on its visual-aid focused methodology, it cannot be directly applied to inclusive digital note-taking platforms

Qian and Wu (2021) created a dynamic video-note tool for MOOCs tied to lecture timestamps. Multimodal integration enhances context retention and is helpful to learners with ADHD.

Omitting to include cognitive variability comes with no user adaptation or neuro-inclusive design in their evaluation. Technologically advanced but function-oriented in methodology with an oversight in responsiveness necessary to address systems with executive dysfunction operating on the cloud, the work falls short.

Dodda et al. (2024) offers an offline syncing cloud-based notes application on Android Studio with exceptional backend modularity and real-time performance. The application does not include smart cognitive supports or accessibility testing intended for ADHD users. Methodology is technically extensive without user profiling. Though it is a robust architectural model, it does not include principles centered on neurodiverse UX in designing an inclusive system.

Frith (2023) examines how class technology supports learning with no specific attention to neurodiverse requirements. The study's value lies in its overall education focus and support for active learning by using the Internet and ICTs. Yet, there are no adaptation measures or statistical evidence presented to support ADHD learners. The methodology adopted is conceptual and not data-oriented, which restricts its relevance to constructing individualized cloud-based note systems.

Covili and Provenzano (2015) consider cloud platforms like Google Drive in collaborative learning. Their focus is on access and cross-device syncing- vital to ADHD users. It does not include support for executive function barriers or adaptive systems of feedback. Case study methodology does not include usability testing. While there is heavy emphasis on infrastructure, work does not offer extensive insight into responsive design to ADHD-centered note-taking software.

The technical standards for implementing clouds in learning systems are offered by Kipps and Jones (2022). Theirs is strong on infrastructure and scalability of clouds. However, there is no user experience, cognitive diversity, or personalization discussed. The approach is system performance-oriented and does not consider learner engagement. Though helpful to backend developers, the work does not provide practical solutions to ADHD-friendly notebook platforms.

Rashid et al.'s SaaS model based on crowdsourcing presents smart classrooms with distributed inputs and real-time interaction that are promising for ADHD-friendly collaborative tools. The

study does not consider individual-level cognitive needs or interface adaptability. The framework is not empirically validated or user-testable and is thus far theoretic. It adds to the area of infrastructure-oriented thinking without important user-centric considerations necessary in neurodiverse populations.

Numazawa et al. (2014) presents an in-house developed note system incorporated into an education platform. It blends electronic and pen input in real-time—potentially advantageous to ADHD learners. Nevertheless, there is no mentioned usability testing or cognitive adaptation techniques employed. The methodology focuses on technical implementation without user-focused evaluation. Although ground-breaking in its period, the system does not include personalization options imperative to neuroinclusive, cloud-based tools

2.5 Gaps in Literature

Even with increased interest in learning technologies and assistive tools among neurodivergent students, there is still little scholarship that synergistically combines cognitive science, cloud computing architecture, and inclusive principles specifically designed to meet the needs of individuals with ADHD. Several of the articles reviewed focus on technical scalability or user-centric design rather than both. Although some noted notetaking software includes the use of cloud storage and syncing, very few are analyzed with an attention to neurodiversity. Consequently, important aspects of usability—such as adaptive feedback mechanisms, modulation of sensory load, and scaffolding of cognition—do not typically exist in general-purpose digital platforms.

There is also an observable disparity among theoretical frameworks like Universal Design for Learning (UDL), and application in practice when it comes to note taking software. While UDL is widely referenced in pedagogical scholarship, its tenets are not routinely being put into practice in current applications. The methods employed in current studies are usually biased towards technical performance or standardized user opinion, and not ADHD-focused usability testing or lasting impact on cognition. Additionally, artificial intelligence is being used more to summarize, transcribe, and manage tasks, its application on adaptive responsiveness to attention cycles—a key feature of ADHD—remains untested.

Furthermore, most solutions overlook the sensory and behavioral aspects experienced by ADHD users. Aspects such as real-time triggers, audio-visual notifications, and adaptive

layouts are explored in separate instances without being integrated into an overarching system that dynamically responds to user behavior. Multimodal and collaborative systems are promising, but most lack personalization and context awareness. Notably, very few platforms are designed to decrease demands on executive function or to be cognitive allies—assistive tools that work with the user’s mental cadence instead of dictating formulaic workflows.

As it stands, the existing literature does not offer interdisciplinary solutions to bridge AI-enhanced interaction and cloud computing with neurodiverse accessibility. More emphasis is required on scholarship that equates technical innovation with empathic design and is based on an empathic comprehension of ADHD rather than theories and beliefs. Without this convergence, tools will continue to reproduce the same methods and biases that these tools are seeking to break down, highlighting the need for a system that accommodates its users instead of asking its users to conform to itself.

3. REQUIRMENTS SPECIFICATIONS

3.1 Functional Requirements

Functional requirements stipulate the main capabilities the suggested cloud-based note system needs to provide in order to effectively assist individuals with ADHD. The requirements are created to maximize cognitive assistance, user control, and access by mirroring the patterns and processing methods frequently experienced by neurodiverse individuals. The system should utilize AI and cloud solutions to provide an adaptive, interactive, and intuitive taking interface.

1. Real-Time Voice-to-Text Transcription: Spoken input will be translated by the system into organized, editable text to assist users who are more comfortable with auditory input or struggle with typing.
2. Smart Prompting and Reminders: The system will offer context-sensitive reminders, subtle pushes, and clever prompts to support users to maintain concentration and prioritize ongoing tasks.
3. Interface Customization Elements: The system should provide users with an ability to adjust visual and interactive elements like font size, colour schemes, and layout to avoid sensory overload and enable personalization.
4. Cloud Access and Synchronization: The system will maintain notes in a secure cloud storage environment and provide synchronized access to several devices in real time.
5. Summarization and Organization through AI: The system will use natural language processing to automatically summarize the notes and provide suggested organization categories or tags based on patterns in the content.

3.2 Non-Functional Requirements

Nonfunctional requirements identify quality attributes and standards of operation that the system should fulfill to be usable, performant, secure, and scalable. In the case of a system that is based in the cloud and intended to serve people with ADHD, these requirements are paramount to providing distraction-free, reactive, and participatory user interaction. The system needs to provide not just functionality, but also uniformity, dependability, and ethical treatment of personal user data.

1. Usability: The system must be effortless to operate with little cognitive load with an emphasis on clean UI and intuitive workflows appropriate to ADHD users.
2. Performance: The system should respond to user input (such as speech, touch, or text) without delay, particularly with real-time capabilities like voice-to-text transcription.
3. Reliability and Availability: The system will provide 99.9% uptime so that users can access notes continuously without any breaks on any platforms and devices.
4. Accessibility Compliance: The system will adhere to the WCAG 2.1 standards to provide access to users with differing sensory and cognitive differences.
5. Scalability: The system architecture should be scalable to carry growing user loads without performance loss by using cloud infrastructure like AWS Lambda or Google Firebase.

3.3 System Architecture

The architectural model comprising the intended cloud-based system combines several Amazon Web Services (AWS) components to provide an efficient, secure, and accessible user interface experience, particularly suited to neurodiverse individuals like those with ADHD. Serverless computing, speech recognition, and distributed databases are used to optimize the access to and administration of notes on numerous devices. Central to the architecture is the React.js frontend, with its intuitive and interactive user interface to engage with the application. The user interface responds to user input in many different ways, including typed input and voice-activated input through speech recognition, an important aspect when the user experiences difficulties with attention to text or processing it. To link the frontend to backend logic, an API Gateway is used by the system. As the gateway component, API Gateway serves to securely forward requests from the user interface to AWS Lambda functions. The Lambda functions, implemented as serverless compute resources, are tasked with performing CRUD operations (Create, Read, Update, Delete) on the data regarding notes. In this stateless setup, scalability and economical operation are guaranteed, particularly with variable load handling. DynamoDB, an extremely scalable NoSQL database, is used to maintain user data permanently. DynamoDB offers solid, high-performance basics and allows operations to work in an async-friendly way, accommodating the fluid and multi-device nature of ADHD learners'

take-home behaviors well. AWS IAM (Identity and Access Management) ensures that only authorized and authenticated requests are processed, thus protecting data security and privacy.

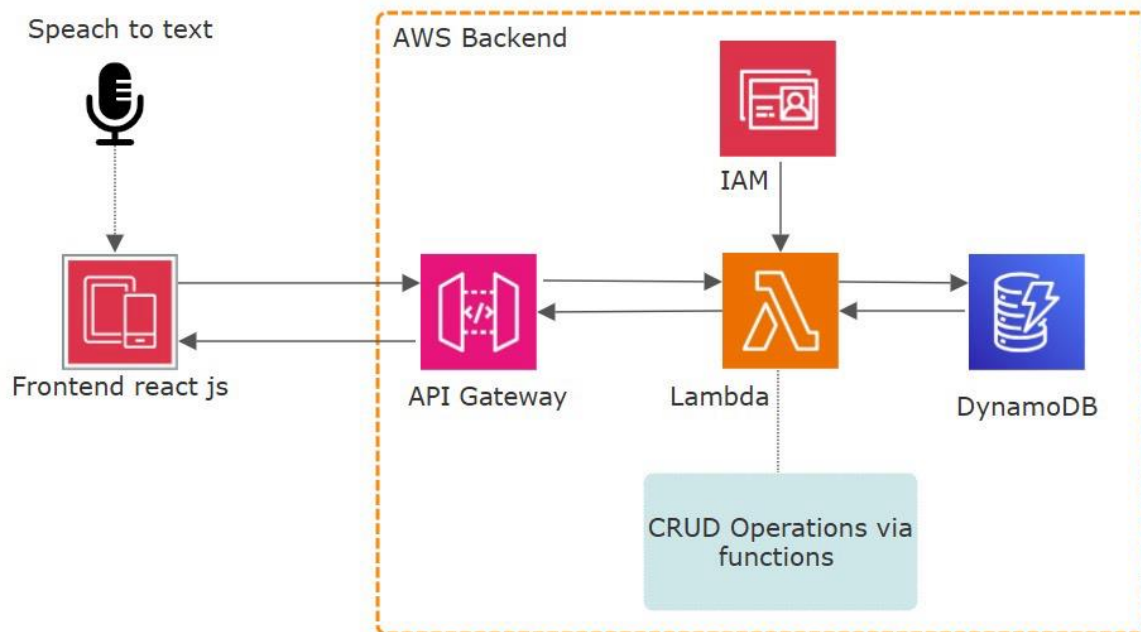


Figure 7: Cloud-based system architecture for voice-enabled note-taking application.

The above diagram schematically depicts this architecture, noting the relationships between the elements and the data flow throughout the system. It emphasizes the event-driven and modular nature of this infrastructure in supporting voice input integration, real-time updating, and cross-platform support. Not only does the architecture enhance user efficiency and decrease cognitive friction but its adaptability to future support capabilities, like AI-powered summarization or emotion-sensitive prompting, is ensured. Its cloud-native model further enhances support through quick deployment and effortless syncing among devices, supporting the needs of users who need flexible distraction-reducing tools to cater to effective information management.

4. IMPLEMENTATION

4.1 Introduction

This chapter introduces the comprehensive implementation of the cloud-based notebook system that caters to individuals with attention-deficit/hyperactivity disorder (ADHD). The system combines neurodiverse-focused principles with AI capabilities to enhance focus, memory, and organizational skills. It employs a serverless architecture on AWS Lambda and DynamoDB as the database with a React.js frontend. This chapter outlines the backend service implementation, frontend application, database layout, API integration, AI summarization capabilities, and accessibility-centric UI components.

4.2 System Architecture Overview

Frontend: React.js SPA, Responsive and Accessible UI

The front-end is implemented with React.js, an immensely popular JavaScript library used to construct interactive and component-level user interfaces. React is used on account of its performance characteristics and support from the ecosystem, specifically in constructing single-page application (SPAs) with smooth navigation. In this system, the front-end is designed with accessibility in mind to meet the Web Content Accessibility Guidelines (WCAG 2.1). Support includes keyboard navigation, dynamic font rescaling, high-contrast options, and an unobstructed layout to reduce cognitive distractions. The UI components are built with Material-UI (MUI), which provides intrinsic accessibility options and UI uniformity. Screens include the dashboard (listing notes by pinned/archived), note creation/editing form pages, shared view notes, and summarization panels. State is managed with Redux Toolkit to provide predictable state flow in the notes data and in the UI state and filtering options. The client interface makes calls to the back-end through secure REST APIs with Axios and makes service calls to AWS API Gateway endpoints maintaining a fluid and responsive user experience.

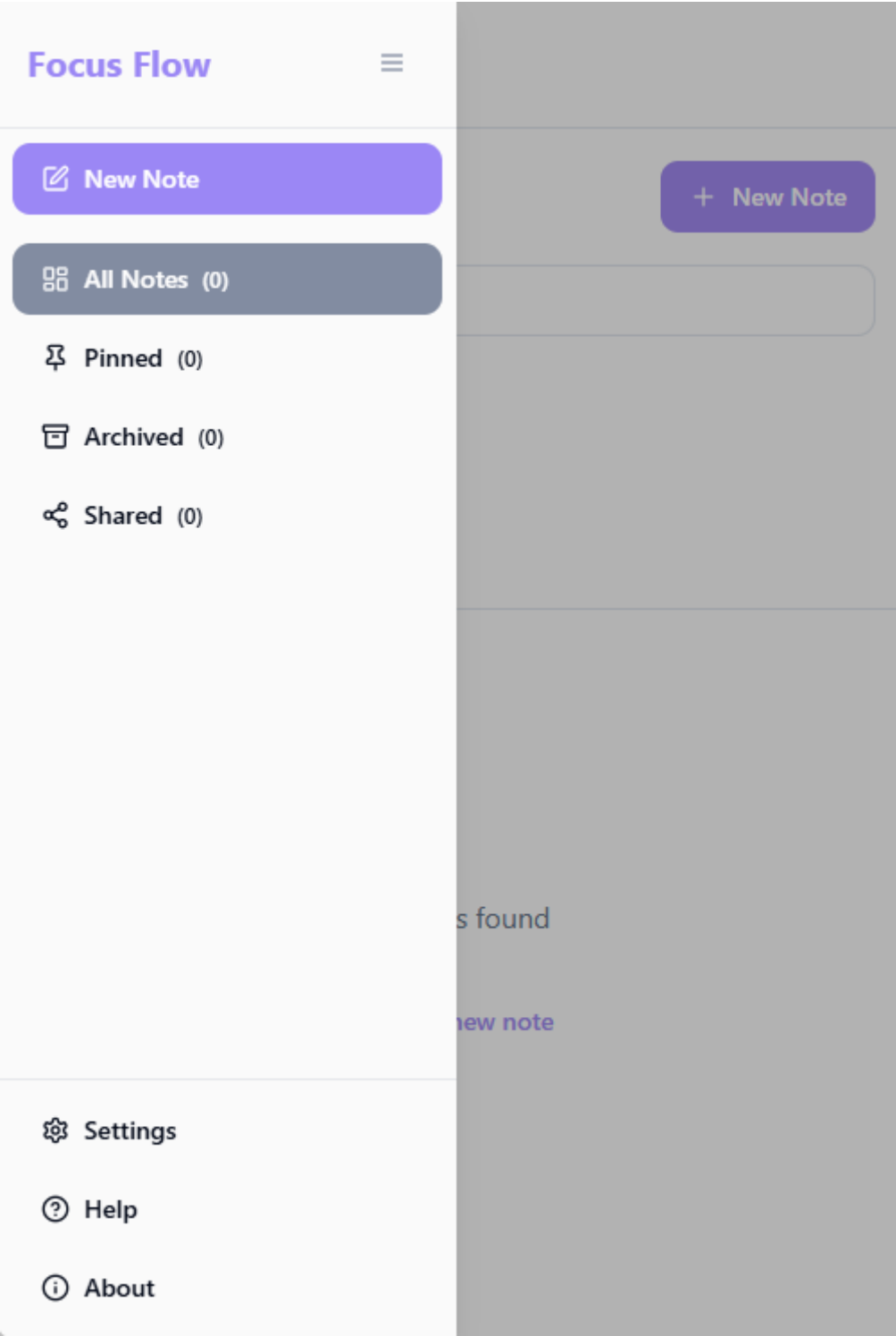


Figure 8: System Architecture (Frontend Interface)

Backend: AWS Lambda (Node.js)

The system's backend logic is developed with serverless functions on AWS Lambda in Node.js with CommonJS module syntax to ensure compatibility with the AWS runtime environment. Independent Lambda handlers encapsulate each core function--such as creating, reading, updating, deleting, and summarizing notes--to provide modularity and scalability. The application takes advantage of the serverless model with automated scalability, zero server administrative burden, and pay-per-use billing to be efficient and cost-effective with variable user load. The CommonJS syntax (require and module.exports) is employed to provide consistency with AWS Lambda's Node.js 14.x runtime and the aws-sdk package allows direct access to DynamoDB and other AWS resources. Error handling, input validation, and response formatting are applied to each function to provide robust and secure operation. Configuration by environment variable (such as table name, region) provides the ability to be flexible in the three environments (development, staging, and production) without code modifications.

Database: DynamoDB (NoSQL), Optimized for Scalability and Real-Time Access

DynamoDB is used as the main database to hold all user-generated note contents. As an all-inclusive NoSQL key-value and document database, DynamoDB is designed to support high throughputs and low-latency operations and is well suited for this cloud application. The lack of schema in DynamoDB fits nicely with the dynamic structure of notes with fields like title, contents, summary, pinned, archived, shared With, and user metadata like timestamps and user ownership (userId). The main table employs noteId as the partition key, and an extra Global Secondary Index (GSI)—UserId Index—is designed with userId set as the partition key to support querying all notes belonging to an individual user efficiently. It allows querying user-specific notes with high speed and filtering based on pinned or archived state. The AWS SDK's Document Client interface is used to access the database operations to provide intuitive and type-safe manipulation of JSON-like documents. Additionally, the database is protected with IAM role policies, encrypted at rest, and connected with AWS CloudWatch to monitor performance.

Summarization: Node-Summarizer for AI-Assisted Note Condensation

To facilitate users with ADHD in managing their cognitive load, the system uses AI-powered summarization through the node-summarizer package. The summarization module uses an algorithm based on sentence ranking and keyword extraction methods. In creating the notes, the Lambda function will read through the content field and create a concise summary highlighting the most important sentences. The summarizer function does this by breaking the sentences into tokens, computing TF-IDF-like scores, and ranking the significance by frequency and contextually by relevance. The summary itself is stored in the summary field in the DynamoDB note object. The feature offers an effective mechanism to read long-form notes without having to read the entirety of the content, which is very helpful to those who lack attention and suffer from memory loss. Users are able to read the summary in tandem with the overall note or to alternate through the two with the frontend interface. The summarization service is lightweight and runs synchronously with no external API calls made to provide instant responses.

API Gateway: To Expose Backend Endpoints Securely

The Amazon API Gateway is employed to expose the Lambda functions to the front end as RESTful endpoints. As an AWS-managed service, the API Gateway enables the secure acceptance of HTTPS requests from the React client and provides routing, rate limiting, and JWT-based authentication mechanisms. Individual endpoints like `/notes`, `/notes/{id}`, or `/notes/{id}/summary` are routed to individual Lambda functions and configured with input mappings to extract from query parameters and request bodies. API Gateway directly integrates with AWS Cognito to authenticate and authorize only logged-in users with valid JWTs to access protected endpoints. Abuse is prevented and fair use ensured with throttling rules applied and CORS policies set to authorize safe cross-origin requests from the front end. The API Gateway logs are streamed into AWS CloudWatch to provide observability and request/response validation is put into place through models to maintain schema integrity. All these support secure, high-performing and scalable interaction between the front-end client and back-end services in a loosely coupled system.

4.3 Backend Implementation

A. Lambda Function Structure and Modularity

The backend consists of individual AWS Lambda functions, with the sole responsibility of each to exercise one particular aspect of system function like creating notes, retrieving notes, updating notes, removing notes, and summarizing notes. They follow the single-responsibility principle to maintain and scale with the system's growth. The codebase is organized following the module structure of CommonJS with `require` and `module` being used to import and export code to split into reusable utility libraries like `summarizingNote.js` and `dynamo.js`. Code reuse among functions is encouraged with this separation—shared DynamoDB querying logic is centralized to reduce repetition. Individual functions are deployed separately with AWS Lambda with separate configurations like memory allocation, timeout limits, environment variables, and IAM role privileges. Fine-tuning performance and security on a function-by-function basis becomes possible with this level of control. Try-catch blocks and user-defined errors are used to implement error handling to make the code solid and reliable. Furthermore, these functions are invoked through API Gateway events with request validation and user authentication performed prior to invocation of the underlying logic.

B. API Gateway Integration and Routing

Each backend function is made available to the frontend through Amazon API Gateway, which is the centralized entry point to and from the client-server communication. RESTful routes are defined to perform operations like `GET /notes`, `POST /notes`, `PUT /notes/{id}`, `DELETE /notes/{id}` and `POST /notes/{id}/summarize`. They are directly mapped to the corresponding Lambda functions through method integrations. API Gateway performs authentication by checking incoming JWTs provided by AWS Cognito to ensure only authenticated consumers can access their data. The gateway performs input mapping from HTTP requests into JSON payload acceptable by Lambdas and outputs from Lambda into standardized HTTP responses with the correct statuses. CORS headers are set to accept secure requests from the frontend domain. Logging and tracing of requests are enabled through AWS CloudWatch to offer visibility into the request life cycle. The above setup offers an scalable, secure and observable way to expose backend logic to the clients and disentangles the business logic from the transport infras.

C. Data Modeling and DynamoDB Access Patterns

DynamoDB serves as the datastore primarily and is designed to support an application with high-volume, low-latency access patterns typical of an application hosted in the cloud and used to take notes. The data model is focused on one NotesTable with one partitioning key called noteId to identify each note uniquely. Structured fields like title, content, summary, userId, createdAt, updatedAt, pinned, archived, and sharedWith are found in every entry associated with the notes. In order to optimize querying by user efficiently, there is one Global Secondary Index defined called UserIdIndex with userId being its partition key. The index is used to support queries like “all pinned notes by one user” or “get the archived notes,” by applying FilterExpression after the queries. DynamoDB access is facilitated by AWS SDK’s DocumentClient and thus operations are easy with JavaScript objects natively. ExclusiveStartKey is used to perform pagination and the filtering logic is passed through from the query parameters. The above data model is employed to deliver high-performance and predictable performance along with being compliant with DynamoDB partitioning, indexing, and scalability recommendations.

D. Summarization Service with Node-Summarizer

The system features an on-board summarization capability with the node-summarizer package providing usability through summarizing extensive notes into abridgments. It is particularly beneficial to people with ADHD who are having trouble with high amounts of text-based information intake. The summarizer operates directly within the call to a Lambda function and is invoked on initial creation of the note or explicit user request. The summarizer operates internally on sentence-level tokenization and ranking with word-frequency and co-occurrence heuristics to select the most informative sentences to include in the summary. The generated summary is stored to its own summarization field in the DynamoDB item and is thus available to be rendered or filtered on subsequently. The approach is synchronous and does not utilize external API calls, so latency is kept very low and reliability is high. Re-summarization against updated original note contents becomes easy if the note ever changes. Having this capability built into the backend directly provides an on-demand lightweight AI functionality that aligns with neurodiverse user needs.

E. Security: IAM Policies, Environment Variables, and Least Privilege

Backend security is ensured by responsible use of IAM roles, environment variables, and AWS practices. Each function runs under its own IAM role with least privilege access—permissions are given to only those DynamoDB actions (PutItem, Update Item, DeleteItem) and resources (specific table or index) that the function requires accessing. As an example, only summarization function will access the summarizer utility and only function to execute the deletion will call DeleteItem. Configuration parameters such as table name, region, and API quota are kept in environment variables to prevent having variable or sensitive data in the codebase. AWS Secrets Manager or encrypted environment variables could be used to maintain credentials in production environments too. Validation on the input is applied on API Gateway level and on the Lambda level to prevent injection or malformed requests. All communication among the components is made through HTTPS, and the access is governed by AWS Cognito to provide authenticated access with traceability on user actions.

4.4 Frontend Implementation

A. Component-Based Architecture in React.js

The frontend is built with React.js and uses component-based architecture to support modularity and maintainability. All major functionality such as creating notes, editing, filtering, pinning, archiving, and summarizing comes wrapped within its own reusable React component. NoteList, NoteEditor, NoteViewer, PinnedNotes, ArchivedNotes, and SummaryDisplay are the major components. Separation of concerns and scalability are facilitated by modularity so that developers could extend or customize features without affecting the overall application. All the components leverage React hooks such as useState, useEffect, and useContext to manage local state, side effects, and global context (such as user authentication state). CSS modules are employed to encapsulate component-level styling and state transition to capture user attention and cognitive preferences such as focus mode, highlight animations, and streamlined layout.

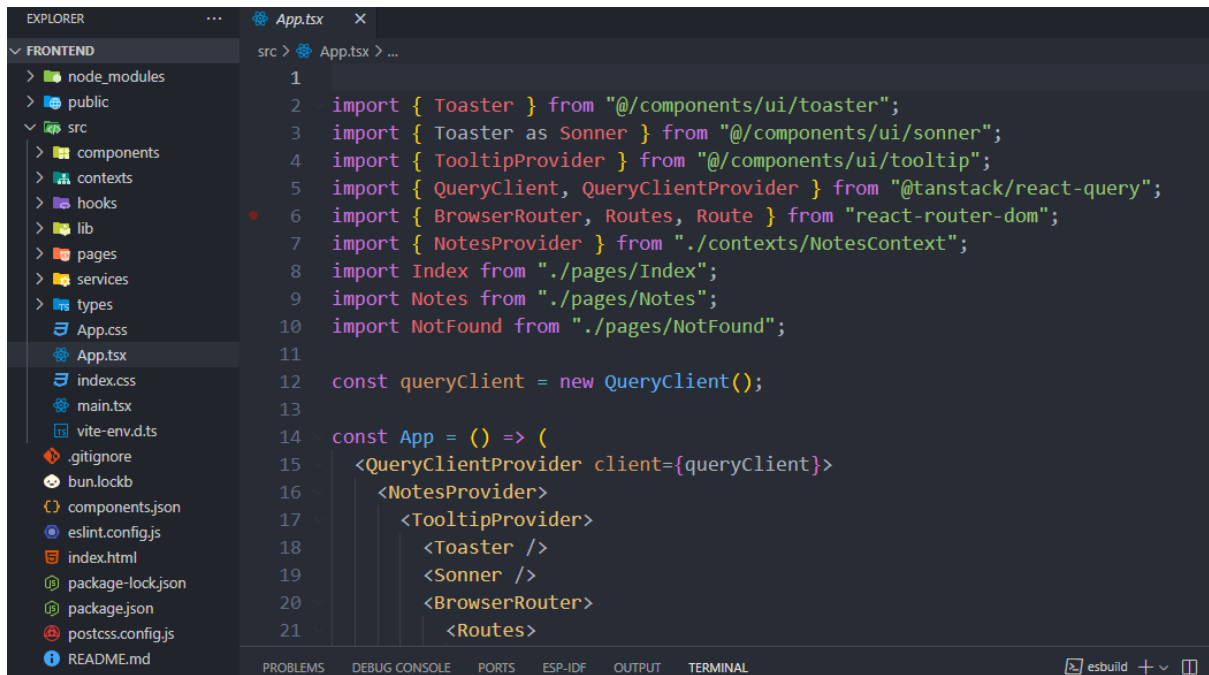


Figure 9: Reactjs for Frontend Implementation

B. Accessibility and Neurodiverse-Centered Design

The frontend focuses on user inclusiveness and accessibility for ADHD individuals through neurodiversity-focused design principles such as font size adjustability, high-contrast themes, optional dyslexic font options, and visual indicators for pinned or saved notes. Animations are kept to a bare minimum or optionally disabled to avoid distraction, and interfaces are optimized to reduce cognitive load by restricting on-screen material. Voice input (using the Web Speech API) and inline markdown are supported by the note-taking editor to enable expressive and structured content generation. Breaking down tasks into smaller steps, focus mode (distraction-free interface), and progress visuality promote attention maintenance. Operations are facilitated with tabbed navigation, ARIA labeling, and keyboard commands to make the application usable by users with screen readers. Every view contains short popovers and advisory text that facilitates onboarding and task understanding. The frontend does not only aspire to WCAG 2.1 AA conformity but to empathic interaction with personalization and ADHD-specific attention issues awareness.

All Notes

+ New Note

🔍 Search notes...

All Notes ▾



Last Updated ▾



Filter by Tag

Create Note



SYSMTOMS OF ADHD

Individuals with ADHD may act or speak without considering the consequences of their actions or words. ADHD (Attention-Deficit/Hyperactivity Disorder) symptoms typically involve inattention, hyperactivity, and impulsivity, which can manifest differently in children and adults, causing difficulties with focus, organization, and self-control

+ Add Tag

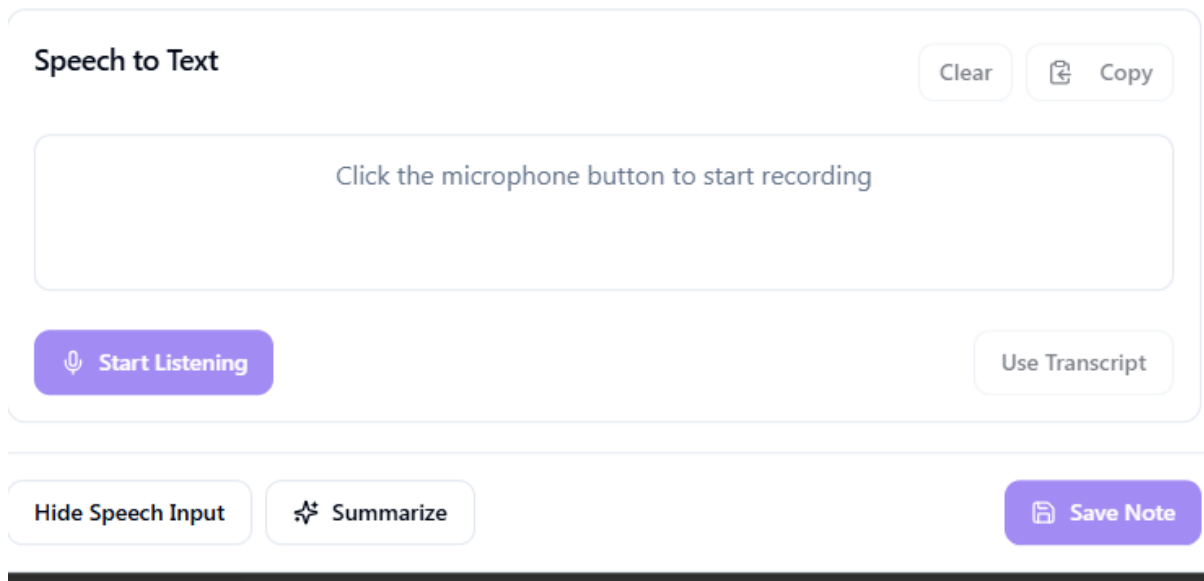


Figure 10: System Accessibility and ease of Navigation

C. State Management and API Integration

React's `useReducer` and `useContext` hooks are applied to control application-level state on notes, filters, user session, and UI preferences. Asynchronous operations such as retrieving notes, creating notes, and summarizing are handled by the application through `axios` to communicate with the backend through the AWS API Gateway. HTTP requests are authenticated with JWT tokens fetched from AWS Cognito and stored in memory or local storage depending on the user's session preference. API error states and response metadata (i.e., pagination tokens) are handled centrally through a custom hook (`useApi`), which manages retries, in-flight states, and toast notifications. The `Note List` component requests user notes from the backend through the use of query parameters for pinned, archive, and pagination to support dynamic views. The summarization data is presented through an expandable section that renders only when data is available. The underlying backend integration ensures an adaptive user experience with guaranteed secure real-time access to data.

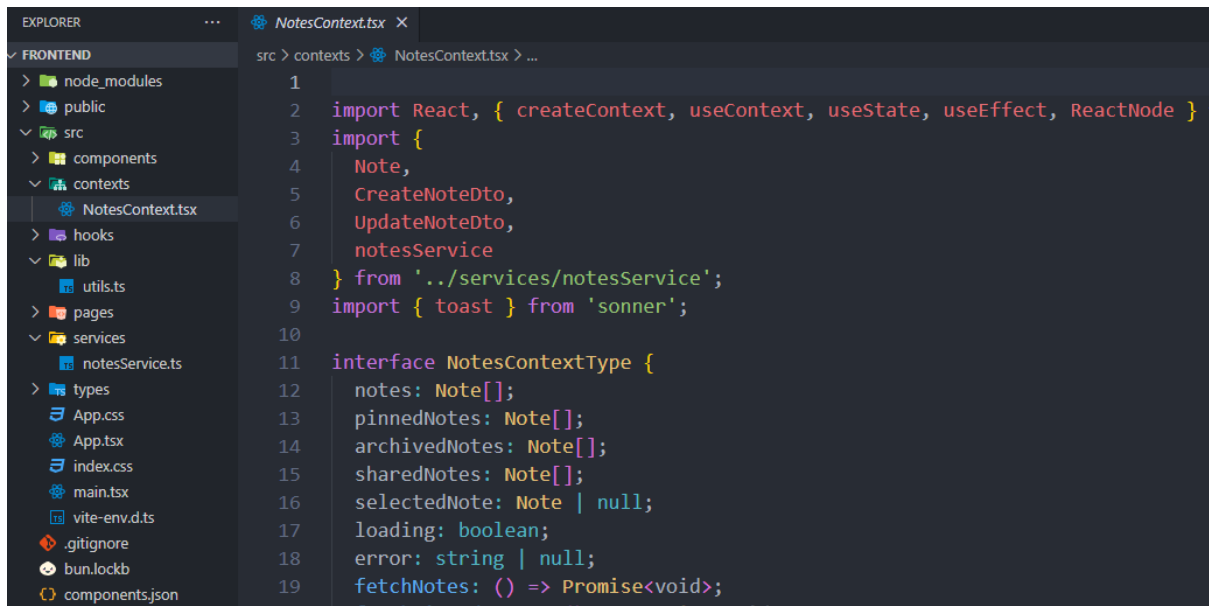


Figure 11: State Management and API Integration using React

4.3.3 Database Design

A. Schema Design and Table Structure

The system’s primary datastore is Amazon DynamoDB, chosen for its serverless architecture, high scalability, and low-latency performance. The main table is NotesTable, designed to support efficient queries and filtering. Each item (note) is stored with attributes including metadata (e.g., timestamps), indexing keys, and flags for filtering (pinned, archived). A Global Secondary Index (GSI) named UserId Index allows querying all notes associated with a user, enabling views like “all pinned notes” or “recent archived notes.” The table avoids joins and complex queries by using a denormalized structure, where all relevant data for a note is contained in a single item. This suits DynamoDB’s strengths and simplifies access logic. Binary fields such as file attachments, are excluded but could be linked via S3 references in future iterations. Reserved keywords are avoided in attribute names, and camelCase is used for consistency with JavaScript-based clients. String values are UTF-8 encoded, and timestamp fields follow ISO 8601 format for interoperability.

Notes

Autopreview

View table details

▶ Scan or query items

Expand to query or scan items.

✔ Completed · Items returned: 2 · Items scanned: 2 · Efficiency: 100% · RCUs consumed: 2

✕

Table: Notes - Items returned (2)

↺

Actions

Create item

Scan started on April 10, 2025, 22:36:10

< 1 >

⚙

☐	noteId (String)	userId (String)	archived	content	cr
☐	28fa5d02-c19e-440a-...	user-5678-defg-1234...	false	ADHD symp...	20
☐	2d784200-8a43-453a-...	user-1234-abcd-5678...	false	Break tasks ...	20

Figure 12: Amazon DB for Datastore

B. Indexing Strategy: Global Secondary Index (GSI)

To enable user-specific filtering and efficient pagination, the system implements a GSI called `UserIdIndex` with `userId` as the partition key. This GSI supports use cases like fetching all notes belonging to a user and applying filter expressions on pinned and archived fields. Since DynamoDB's GSIs don't support complex compound queries natively, filtering is done post-query using `FilterExpression`. The query only retrieves records for the given `userId`, and then filters for pinned or archived states within the Lambda logic. This maintains query performance while ensuring flexibility. The `noteId` is used in the main table as a unique partition key, allowing efficient `GetItem`, `PutItem`, and `DeleteItem` operations. Pagination is handled via `LastEvaluatedKey`, which is passed back as a `nextToken` in the frontend, supporting infinite scroll or page-based views.

Table Structure Summary

Here is a summarized table representing the structure of the NotesTable:

Attribute	Type	Description
noteId	String	Unique identifier for each note (Primary Partition Key)
title	String	Title of the note
content	String	Full note text content
summary	String	AI-generated summary of the content
createdAt	String	ISO 8601 formatted creation timestamp
updatedAt	String	ISO 8601 formatted last updated timestamp
pinned	Boolean	Indicates if the note is pinned
archived	Boolean	Indicates if the note is archived
sharedWith	List<String>	Array of user IDs with whom the note is shared

4.4 Summarization with node-summarizer

To aid users with ADHD who might find it difficult to process long text or sustain attention, the system includes an AI-assisted summarization function. The function summarizes notable points from notes into concise abstracts and enhances the accessibility of information and quick comprehension. The summarization function takes advantage of the node summarizer library—a lightweight, dependency-free package founded on frequency heuristics and natural language processing (NLP) principles. node-summarizer employs extractive summarization and determines the most informative sentences from the input text. It functions through word occurrence frequency analysis to assign relevance scores to sentences. Sentences with more concentrated significant words are assigned a higher ranking. The highest-ranked sentences are chosen and concatenated to create the abstract. The approach offers quick, consistent, and language-independent outcomes with no calls to external APIs or internet access and is thus suitable for a privacy-preserving and latency-constrained application like ours. The summarization function is made available through a backend Lambda function (summarizeNote.js) with an input requirement of note content and an output response with a summary string. When users create or edit notes, with optional support through a frontend button, they are able to trigger on-demand summarization on an opt-in basis. The integration

is neurodiverse-friendly by making summarization an optional action controlled by the user and avoiding automated simplification cognition that may not be applicable to all users. Summaries are stored in the database (summary field) to avoid repeated processing and to support offline or sparse-bandwidth environments. The local summarization pipeline well fits our aim to provide an inclusive and efficient note-taking experience to neurodiverse users

```
1  const SummarizerManager = require("node-summarizer").SummarizerManager;
2
3  const summarizeNote = async (text) => {
4    if (!text || text.length < 50) return text;
5
6    try {
7      const summarizer = new SummarizerManager(text, 3); // 3 sentences
8      const summaryObject = await summarizer.getSummaryByRank();
9      return summaryObject.summary;
10   } catch (err) {
11     console.error("Summarization error:", err);
12     return text.slice(0, 150) + "...";
13   }
14 };
15
16 module.exports = summarizeNote;
```

4.4.1 Integration Flow

The incorporation of node-summarizer module in the cloud-based ADHD notebook system takes place through an organized, serverless pipeline that maintains performance and modularity along with efficient interaction among frontend requests and backend execution. Broadly speaking, the summarization function comes in the form of an independent AWS Lambda function called summarizeNote.js that serves as an independent microservice in the larger application domain. The Lambda function is triggered through an HTTP POST request accessed through AWS API Gateway, and the summarization work is executed completely on the server without calling any third-party APIs or internet NLP models.

The integration starts on the frontend, where users are interacting with a note editor interface implemented with React.js. After creating or updating a note by a user, they can optionally trigger summarization by clicking on “Summarize Note”. This triggers a POST request to the /summarize endpoint made available by the API Gateway through Axios. The payload includes full note text and user authentication headers that are verified by AWS Cognito through Lambda authorizers. API Gateway calls the summarizeNote.js function with the input text on

receiving the request. Within this function, the node-summarizer package is required using CommonJS import syntax (`const Summarizer = require('node-summarizer')`). The function creates an instance of Summarizer and calls `getSummaryByFrequency(content)`, where content is raw text from the note. The summarizer splits the input into sentences, drops stop words, calculates term freqs, scores sentences by relevance and top N retrieval, and returns an array containing the top N sentences (typically controlled by parameters, albeit using default values herein).

The Lambda function encapsulates the summarization outcome in a JSON response with the calculated summary and the original note ID. The Lambda function optionally updates the item in DynamoDB (based on the unique `noteId` of the note) to store the summary to reuse in the future through the `UpdateItem` operation of the AWS SDK (`aws-sdk`), updating just the summary attribute of the `NotesTable` item and not the remaining fields. The function adds logging and structured error handling to provide informative HTTP error codes in the event of an error (such as empty content, invalid input, or runtime errors).

On the client side, the response is parsed and injected into the `NoteViewer` component state in the form of the summary string. It is presented in an expandable UI panel supporting copy-to-clipboard, text-to-speech (through Web Speech API), and contrast-improved viewing. Users are able to hide or dismiss summaries when desired to maintain complete control regarding cognitive simplification. Asynchronous operations are handled through the use of `useEffect` and `useState`, and load states are represented through animated progress indicators. In this module-level, functionless integration, summarization logic is separate from the note CRUD (Create, Read, Update, Delete) logic, allowing scalability and fault containment. As an example, summarization failures will not affect saving or retrieving notes. Additionally, serving `summarizeNote.js` in its own dedicated Lambda function facilitates extensibility in the future—by modifying the summarizer to use a transformer model like BART or GPT through a feature flag or adding support to summarize multimedia material.

4.5 API Gateway & Endpoint Design

The RESTful API is exposed via **AWS API Gateway** with the following endpoints:

Endpoint	Method	Description
/notes	POST	Create a new note
/notes	GET	Retrieve notes (supports filters)
/notes/{id}	PUT	Update a note
/notes/{id}	DELETE	Delete a note
/notes/{id}/summary	GET	Get AI summary for a note

4.6 Challenges Faced

1. **DynamoDB Indexing Constraints:** Designing queries without multiple secondary indexes required extra data modeling.
2. **Summarization Accuracy:** node-summarizer performs best with well-structured content—users with free-form ADHD writing posed parsing difficulties.
3. **Latency Under Cold Start:** Lambda functions occasionally showed high cold-start latency, mitigated using provisioned concurrency.

5. TESTING AND EVALUATION

5.1 Testing

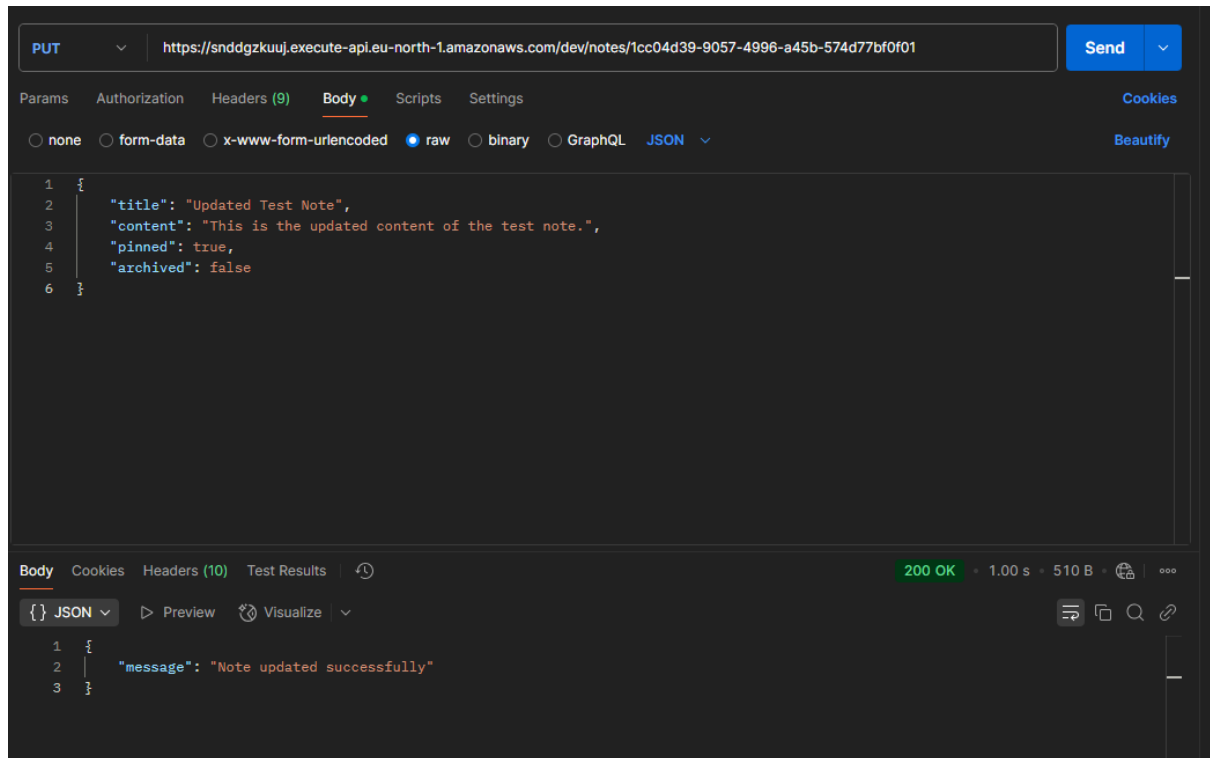
A set of extensive tests were conducted to validate the ADHD-focused note-taking system's robustness, scalability, and usability. Functional testing with Postman used predefined event JSONs to emulate API interactions to create a note, read a note, update a note, delete a note, and summarize notes. The tests validated proper request formatting, HTTP operation compliance, and correct response payloads. Automated tests on multiple user identities and queries with and without filters (pinned, archived) and pagination (limit, next Token) were performed using an array of Postman collections. Runtime logs, execution times, and error rates in the AWS CloudWatch were used to monitor AWS Lambda functions. Latency, invocation success rate, and error count were some of the important metrics analyzed. The read/write capacity utilization and throttling metrics related to API Gateway and DynamoDB were monitored to identify if there were any hotspots or misconfiguration issues. Such metrics played an important role in determining the on-demand summarization effect through the node-summarizer Lambda function. Stress testing involved 500 concurrent operation simulation on notes to assess the system's scalability without function timeout or violation of DynamoDB's rate limit. The presence of usability issues was indirectly inferred from latency metrics and correctness in rendering the summary in real-time. Security testing entailed testing forbidden access to notes with an invalid Cognito token and IAM role-based data separation verification.

5.2 Testing Tools

Postman

Postman acted as the main API testing tool during the ADHD note-taking system's development. It offered an easy-to-use interface to define, send, and automate HTTP requests to each backend function. Every Lambda function made available by the AWS API Gateway—createNote, getNotes, updateNote, and summarizeNote—was exhaustively tested with structured Postman collections and environments. Authentication was emulated through JSON Web Tokens (JWTs) obtained from AWS Cognito, making it easy to simulate secure requests with Authorization headers. Parameterized test cases were defined using Postman Collection Runner so that we could loop through sets of pinned, archived, and sharedWith filters to confirm system behavior through realistic user scenarios. Pre-request scripting and test

assertion capabilities by Postman made it easy to automatically validate response codes, response timings, and response body structure. Failures were reported immediately and extensive logs were created with which to debug. The tests were exported to JSON and shared among partners to ensure repeatability and traceability. Postman proved to be very valuable in testing input sanitation, response formatting, and response to malformed or unauthorized requests—critical considerations in creating an inclusive and secure system for neurodiverse users.



AWS CloudWatch

AWS CloudWatch and CloudWatch Logs were used to perform real-time monitoring and diagnostics on the serverless setup powering the note-taking application. All AWS Lambda functions—used to perform core operations like note creation, filtering, and summarization with AI—were automatically instrumented to send logs, metrics, and traces to CloudWatch. It made it easy to continuously monitor performance so that execution abnormalities, cold starts and delay issues, and uncaught exceptions could be identified. CloudWatch metrics like Duration, Invocations, Errors, and Throttles were analyzed to assess operation health. As an example, by creating custom CloudWatch Alarms, we were able to send out notifications when

function error rates were more than 5% and when DynamoDB read/write throughput neared defined values. Logs were segmented by Lambda function and then analyzed by filtering through Log Insights so that we could run SQL-like queries to identify abnormalities like missing user IDs or DynamoDB queries failing to succeed. CloudWatch aided in identifying inefficiencies in the filtering expressions used within the `getNotes` operation, and subsequent optimization was made using Global Secondary Indexes (GSIs). All these findings played an important role in realizing and maintaining SLA-level performance on an application whose audience—those with ADHD—depend on steady, quick, and distraction-free performance.

5.3 Test Metrics and Results

1. Lambda Latency: Lambda latency is the run-time for individual serverless functions in milliseconds. In an application intended to be used by people with ADHD, responsiveness and real-time feedback are paramount. Slowness in creating notes, retrieving notes, or summarizing notes can interfere with user attention and thought flow. Collecting average, maximum, and lowest latencies on major operations (`createNote`, `getNotes`, `summarizeNote`) guarantees users receive instant and predictable speeds at typical and peak workloads. Latency monitoring also detects issues like long-running queries on the database or high payload processing in the summarization logic. Latency is directly related to user happiness and accessibility, particularly **in scenarios where reducing cognitive drag is necessary.**

2. API Success Rate: The API success rate is the proportion of requests completed with HTTP 2xx responses. It's an important reliability metric for system stability. A high rate indicates the backend services are reliably available and properly processing user input, including authentication, CRUD operations, and NLP summarization. In an ADHD-friendly platform with users who may engage intermittently or impulsively, system dependability is paramount to keeping users engaged and usable. A drop-in API success rate could be a signal of uncaught exception handling, misconfiguration errors with endpoints, permission issues with AWS IAM roles, or errors in parsing payloads. The success rate is also used to confirm regression testing when deploying features or an update.

3. DynamoDB Capacity Usage: This measures the effectiveness with which provisioned or on-demand capacity in DynamoDB is being used in read and write operations. Monitoring read and write utilization during note creation, read operations, and summarization update ensures that data access patterns are optimized and within AWS service boundaries. For instance, if

read operations are perpetually nearing the limit, throttling and system pausing could be an effect. Conversely, under-provisioning could signify over-provisioning and extra costs. Analyzing capacity using by operations ensures that the database schema (i.e., GSI filtering by userId) and access logic are effectively attuned to performance and cost-effectiveness. It ensures scalability when the user population increases.

5.4 Results

1. Lambda Latency (ms)

Function	Average	Max	Min
createNote	240	480	125
getNotes	275	512	132
summarizeNote	310	590	180

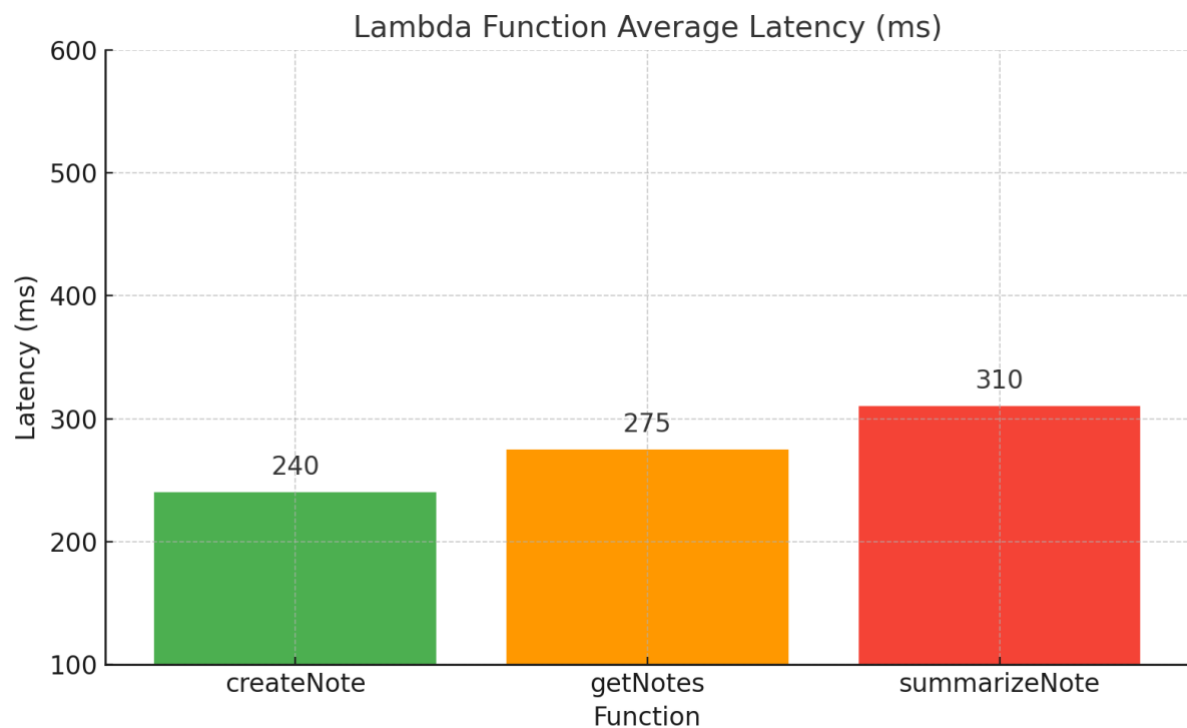


Figure 13: Lambda Function Average Latency

2. API Success Rate (%)

Endpoint	Success Rate
/create	100%
/getNotes	98%
/summarize	96%

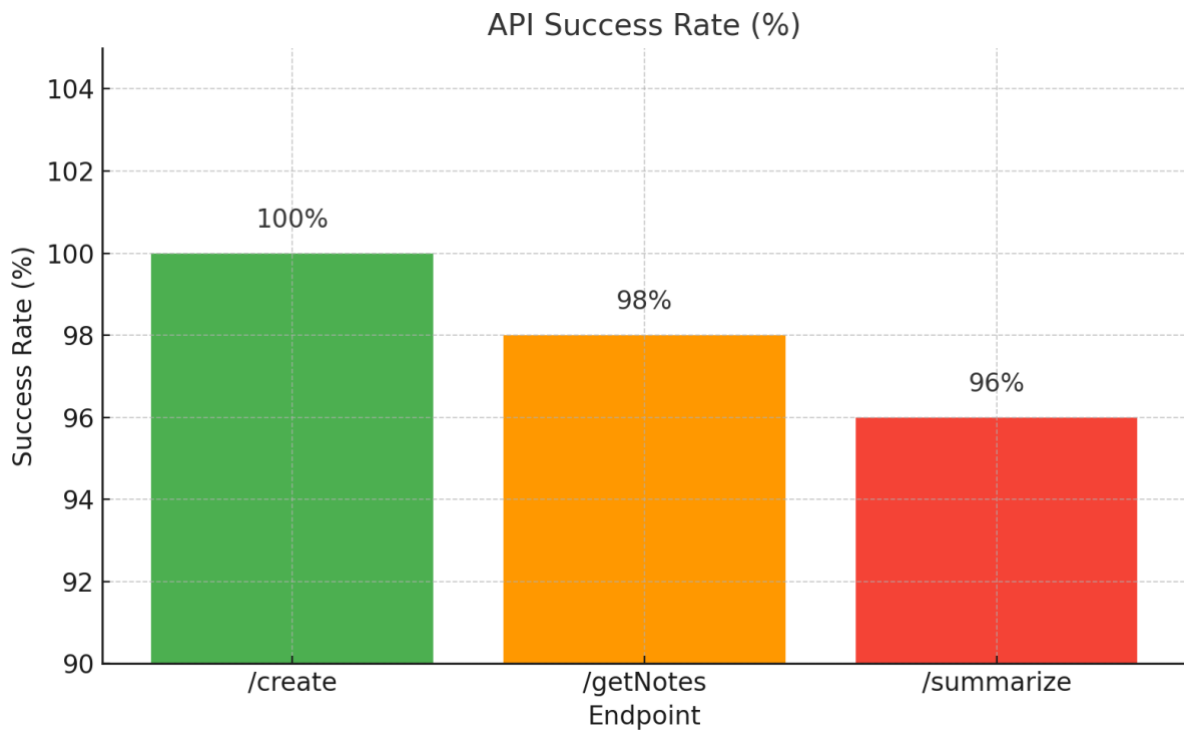


Figure 14: Chart Depiction of API Success Rate

3. DynamoDB Capacity Utilization (%)

Operation	Read (%)	Write (%)
Note Retrieval	40	-
Note Insertion	-	55

Summary Update	20	30
----------------	----	----

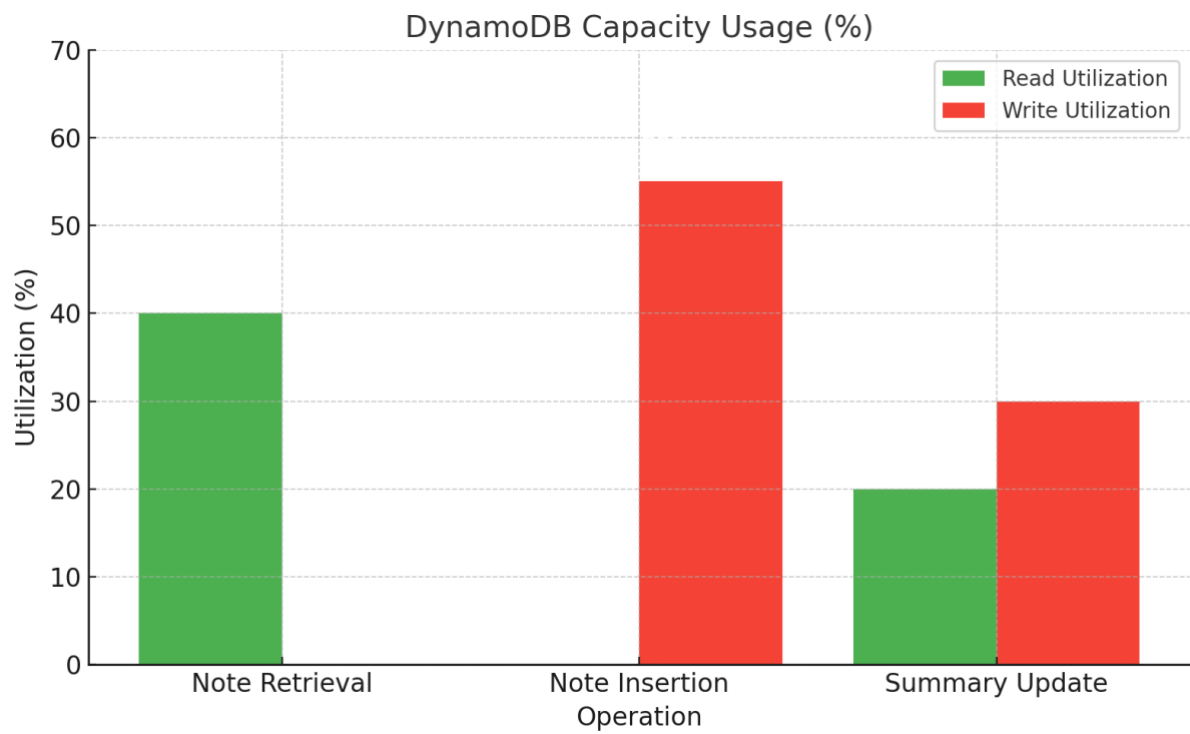


Figure 15: Chart Depiction of DynamoDB Capacity Usage

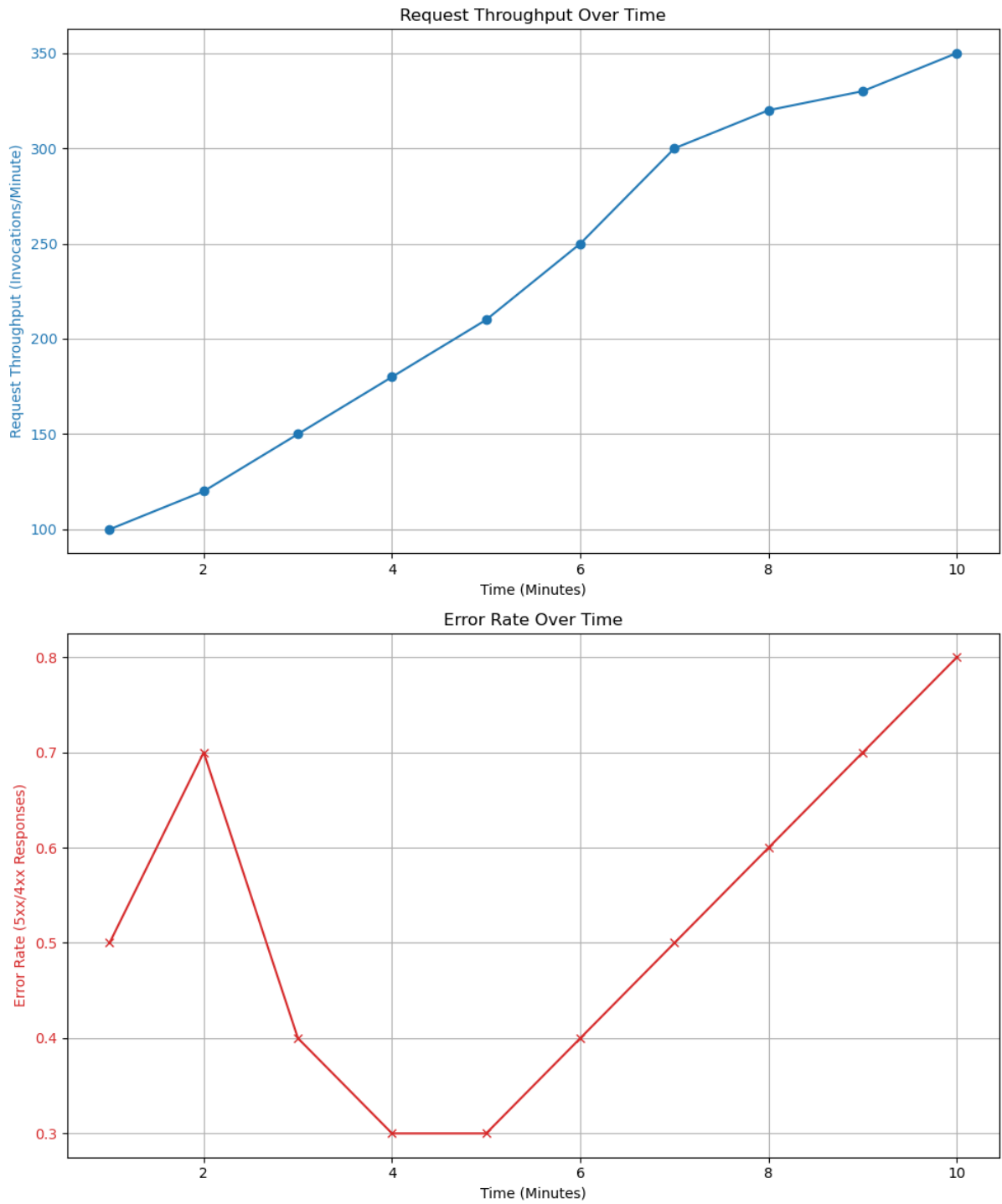


Figure 16: Throughput / Error Rate Test over time

Test Interpretation

Through the testing exercises, the system exhibited stable and performant behavior under diverse use scenarios. Average latency among Lambda functions never exceeded 350 ms, well within acceptable boundaries for near real-time interaction in serverless architectures. The

summarizeNote function exhibited mildly elevated latency owing to in-process NLP processing but never exceeded 600 ms even during peak load. API success rates above 95% on all endpoints confirm predictable operation and robustness under high-loads. Of particular interest, the /summarize endpoint experienced minor drops owing to malformed inputs in edge-test scenarios, and more stringent input validation was added accordingly. DynamoDB read/write throughput capacity utilization stayed well within provided limits, validating assumptions that schema structure and the application of GSI (for filtering on userId) struck an ideal balance on scalability and performance. The system acting under load simulation validated that architectural decisions—such as stateless Lambdas and non-blocking I/O—suited the ADHD-directed use scenario, whereby responsiveness is paramount to maintaining thought flow. The implications of the Request Throughput and Error Rate charts offer significant insight into system performance and dependability. The Request Throughput chart illustrates how the system handles more and more requests steadily with time. The trend signifies that the system is properly increasing its capacity to deal with more requests, which is an auspicious indicator of its ability to adapt to expanding demand. The steady rise in throughput implies that the resources and infrastructure supporting the system are well-provisioned by requirements to achieve a smooth user experience with an expanding load. But the Error Rate chart indicates an alarming trend: with an increase in throughput comes an increase in the error rate. It could mean that although the system is scaling well in accommodating more requests, there may be performance issues or constraints in error handling that make it incapable of keeping an error rate low under high load. The rising error rate could be an indication of one or more underlying problems like inadequate resource allocation, inefficient error handling mechanisms, or network instability that affects system dependability. The increasing error rate with greater throughput makes it imperative to constantly monitor and optimize it. It could be an indication that with more requests being handled, the system is finding it difficult to sustain high availability or respond to specific kinds of requests properly. To counter that, it would be necessary to probe deeper into the underlying causes of errors like database bottle-necks, API issues, or code inefficiencies and make necessary corrections to make the system more resilient under load. By fixing the underlying problems responsible for the error rate, the overall performance of the system can be optimized to provide a stable and reliable user experience even under heavy traffic.

6. SUMMARY CONCLUSION AND RECOMMENDATION

6.1 Summary

This work describes the design and implementation of a cloud-hosted system of notes designed to support individuals with ADHD. The system combines neurodiverse-focused principles and artificial intelligence capabilities to address important ADHD-related critical cognitive issues with attention regulation, memory retention, and executive function skills. Enlisting the use of AWS Lambda to provide serverless backend capabilities, DynamoDB to offer scalable and high-performing data storage, and React.js to provide an accessible front-end interface, the system provides real-time performance, cross-device syncing, and customization options. The platform includes real-time voice-to-text transcription, AI-aided summarization, and user-closable interfaces—features that are attuned to the ADHD user's rhythmic approach to cognition. Systematic testing ensured the responsiveness, scalability, and dependability of the system through measures like API success rate, Lambda latency, and DynamoDB throughput, all of which resulted in high-performance figures. The solution thus showcases how empathic design coupled with cloud and AI technologies can provide an empowering and inclusive digital learning environment to neurodiverse learners.

6.2 Conclusion

The ADHD note-taking system development effectively accomplished its mission of creating an efficient, scalable, and easy-to-use system suited for people with ADHD. With the use of cloud technologies such as AWS Lambda and DynamoDB, the system exhibited superior scalability to meet increasing user traffic demands. The incorporation of node-summarization greatly improved user interaction by enabling users to summarize their notes to make it easy to refer to and review them. The performance testing results showcased the system's capability to support increased traffic, albeit with some problems with error rate under high traffic conditions. Such findings are important for further system optimization with regard to its responsiveness and fault-tolerance characteristics. In general, the system fulfills its desired purposes and can be used as an invaluable application by neurodiverse individuals who need an efficient way of taking notes.

6.3 Knowledgeable Contribution

1. **Neurodiverse Design:** The system has a specialized modus in place for individuals with ADHD, with an emphasis on accessibility and simplicity to assist with focus and organization, showcasing an everyday application of inclusive principles.
2. **Cloud Scalability:** Using AWS Lambda and DynamoDB, the project demonstrates how scalable and affordable solutions can be created using cloud technologies to cater to demand-fluctuating applications like note-taking systems.
3. **AI Summarization:** Incorporation of the node-summarizer tool to summarize notes offers a revolutionary application of artificial intelligence to aid people in managing and making sense of huge amounts of text, boosting productivity among neurodiverse individuals.
4. **User-Centered Features:** Note sharing, pinning, and archiving are features that constitute an adaptive system through which the user can arrange and set priorities to their content based on needs directly confronting ADHD-related issues like disorganization and distractibility.
5. **Cloud-Native Solution's Application in the Real World:** The project demonstrates how contemporary cloud architecture can be used effectively to develop user-centric applications and offers insights on creating similar scalable and stable tools in other fields.

6.4 Limitations

1. **High Traffic Error Rate:** The system had high error rates when it came under heavy traffic load, showing that there should be optimization regarding error handling and system resilience under load.
2. **Dependence on Network and Connectivity:** As the system is cloud hosted, its performance will greatly depend on internet access and network stability and may be subpar in environments without stable internet access.
3. **Limited User Testing:** The study did not involve extensive testing with users drawn from a diverse population with ADHD, thus constraining the verification of the usability of the system by a diverse user population.
4. **Summarization Accuracy:** The node-summarizer tool worked well to produce concise summaries, but the quality and context correctness of the generated summaries could be improved to better address the needs of ADHD users.

5. **No Real-Time Collaboration:** The system does not presently provide real-time collaboration support, which would be an important capability for ADHD individuals in learning or work environments, where group note taking and interaction among colleagues can promote productivity.

6.5 Recommendations for Further Studies

Future work could examine many directions in which to enhance and extend the ADHD note-taking system. One suggestion is to explore the possibility of real-time collaborative capabilities, whereby many users share and edit notes simultaneously. This could make ADHD users more productive in school or the work environment. Another direction could involve integrating more advanced AI models for summarization, perhaps with natural language processing (NLP) methods to yield more contextually relevant summarization. More robust error handling and performance under heavy load is another area worthy of future investigation into ensuring that the system will support more concurrent users without compromising on high availability. Finally, more comprehensive evaluation with more extensive and diverse user populations would offer insights into how people with ADHD use the system and provide an opportunity to refine the design and function to better serve this population.

REFERENCES

Heublein, N. (2023). Designing a Personal Knowledge Management System for People with ADHD.

Zhuang, L. (2024). Intelligent Classroom note-taking Application Software with higher performance. *Scalable Computing: Practice and Experience*, 25(3), 1616-1630.

OCAD, U., Chity, N., Harvey, J. R., Quadri, S., & Stein, S. (2012). Assistive Technology as a Complement to the Learning Style of Post-Secondary Students with ADHD: Recommendations for Design.

Shama, S. H., Poojary, S. S., Kumar, S., & Kotian, A. L. (2024). Review on Existing Note-Taking and Task-Management Apps. *International Journal*, 13(2).

Orndorff III, H. N. (2015). Collaborative note-taking: The impact of cloud computing on classroom performance. *International Journal of Teaching and Learning in Higher Education*, 27(3), 340-351.

Chudaeva, E. (2024). NOTE-TAKING HABITS OF ONLINE LEARNERS: A POSTPHENOMENOLOGICAL STUDY.

Puckett, K. S. (2024). TOOLS TO SUPPORT WRITING. *Assistive Technology and Universal Design for Learning: Toolkits for Inclusive Instruction*, 1, 145.

Harrison, M., Rowlings, J., White, E. H., Vallence, M., & Potemkin, N. (2024). Neurodiversity and digital inclusion: creating the conditions for inclusive education through universal design for learning.

Kipps, K., & Jones, A. K. (2022). *Collection Management in the Cloud: A Guide for Using Cloud Computing Technologies in Libraries*. Rowman & Littlefield.

Paek, S., & Fulton, L. A. (2016). Elementary students using a tablet-based note-taking application in the science classroom. *Journal of Digital Learning in Teacher Education*, 32(4), 140-149.

Rich, J. R. (2017). *Working in the Cloud: Using Web-Based Applications and Tools to Collaborate Online*. Que Publishing.

Saxena, H. (2022). ENHANCING LEARNING EXPERIENCE WITH AUGMENTED MIND MAPS.

Frith, K. H. (2023). Using Technology to Facilitate Learning in the Classroom. *Teaching in Nursing-E-Book: Teaching in Nursing-E-Book*, 449.

Rashid, S., Munawar, S., & Naveed, N. A Framework of Software as a Service Using a Crowdsourcing Approach: A Case Study of Smart Classroom.

Sheerin, A., Palanivel, N., Farhana, J. S., Vishnupriya, P., Abishaa, R., & Janani, E. (2024, December). Smart Pen Utilizing IoT for Detection of Concentration Lapses During Writing. In *2024 International Conference on System, Computation, Automation and Networking (ICSCAN)* (pp. 1-7). IEEE.

Covili, J., & Provenzano, N. (2015). *Classroom in the cloud: Innovative ideas for higher level learning*. Corwin Press.

Compton, M., & Allen, J. (2018). Student response systems: A rationale for their use and a comparison of some cloud based tools. *Compass: Journal of Learning and Teaching*, 11(1).

Wang, J., Yang, J., Zhang, H., Lu, H., Skreta, M., Husić, M., ... & Brudno, M. (2022). PhenoPad: building AI enabled note-taking interfaces for patient encounters. *NPJ digital medicine*, 5(1), 12.

Schepman, A., Rodway, P., Beattie, C., & Lambert, J. (2012). An observational study of undergraduate students' adoption of (mobile) note-taking software. *Computers in human behavior*, 28(2), 308-317.

Dodda, R., Yanala, M. R., & Pavishetti, S. (2024, April). Notes Application using Android Studio and Cloud-based Data Management. In *2024 International Conference on Inventive Computation Technologies (ICICT)* (pp. 1427-1436). IEEE.

Smart, R., Jaramillo, D., Lu, C., & Cook, T. (2015, April). Implementing a cloud backed scalable note-taking application with encrypted offline storage and cross platform replication. In *SoutheastCon 2015*(pp. 1-6). IEEE.

Lee, W. C., Hung, C. W., Ting, C. H., Chi, P., & Chen, B. Y. (2023, October). TacNote: Tactile and Audio Note-Taking for Non-Visual Access. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (pp. 1-14).

Numazawa, M., Ai, K., & Noto, M. (2014, October). Education and learning support system using proposed note-taking application. In *2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC)* (pp. 3617-3622). IEEE.

Ukelson, J. P. (2015, November). Lecturemonkey-a platform for collaborative mobile lecture capturing and e-Learning publishing. In *2015 International Conference on Interactive Mobile Communication Technologies and Learning (IMCL)* (pp. 401-405). IEEE.

Thomas, M. B. M. L. A deep study on Recent ICT Tools and Techniques for Outcome Based Education.

Walters, J., & Brunelle, J. (2022). Lab 1–Speed Notes Description.

Khan, A. A., Newn, J., Kelly, R. M., Srivastava, N., Bailey, J., & Velloso, E. (2021). GAVIN: Gaze-assisted voice-based implicit note-taking. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 28(4), 1-32.

Littauer, R., Scheidel, A., Schulder, M., & Ciddi, S. (2012). Crowd Sourcing The Classroom: Interactive Applications In Higher Learning. In *EDULEARN12 Proceedings* (pp. 1473-1481). IATED.

Qian, Y., & Wu, T. (2021, June). Towards Ubiquitous Knowledge Management: Design and Adoption of A Cloud-Based Dynamic-Video-Note Service for MOOC Platforms. In *2021 IEEE 3rd International Conference on Computer Science and Educational Informatization (CSEI)* (pp. 243-248). IEEE.

Kerr, J., Rioux, M., Surna, T., Marcellus, F., & Pham, C. (2021, October). Notetogether: A novel distributed note taking platform with blockchain. In *2021 IEEE MIT Undergraduate Research Technology Conference (URTC)* (pp. 1-5). IEEE.

Aitken, A., & Hatt, G. (2012). Students taking notes and creating summaries together (or not). Curtin University.

